

Universal Composable Transaction Serialization with Order Fairness

Michele Ciampi¹ , Aggelos Kiayias² and Yu Shen³ 

¹University of Edinburgh, michele.ciampi@ed.ac.uk

²University of Edinburgh and IOG, aggelos.kiayias@ed.ac.uk

³University of Edinburgh, shenyu.tcv@gmail.com

Abstract

Order fairness in the context of distributed ledgers has received recently significant attention due to a range of attacks that exploit the reordering and adaptive injection of transactions (violating what is known as “input causality”). To address such concerns an array of definitions for order fairness has been put forth together with impossibility and feasibility results highlighting the difficulty and multifaceted nature of fairness in transaction serialization. Motivated by this we present a comprehensive modeling of order fairness capitalizing on the universal composition (UC) setting. Our results capture the different flavors of sender order fairness and input causality (which is arguably one of the most critical aspects of ledger transaction processing with respect to serialization attacks) and we parametrically illustrate what are the limits of feasibility for realistic constructions via an impossibility result. Our positive result, a novel distributed ledger protocol utilizing trusted enclaves, complements tightly our impossibility result, hence providing an *optimal* sender order fairness ledger construction that is also eminently practical.

Contents

1	Introduction	3
1.1	Our Results	4
2	Preliminaries	8
3	Ledger Functionality with Parametric Extend Policy	9
4	Extend Policies with Sender Order Fairness	13
4.1	Sender Order Fairness	13
4.2	Approximate Sender Order Fairness	15
5	Protocol Details	18
5.1	Transaction Encryption from Trusted Hardware	18
5.2	Blind Transaction Serialization	23
5.3	Further Discussions	25
5.4	A Full Protocol Description	25
6	Security Analysis	32
6.1	Blockchain Security Properties	36
6.2	Composable Guarantees	37
A	Preliminaries (Cont'd)	42
B	Glossary	47

1 Introduction

An important variant of the consensus problem [PSL80] that has recently gained significant attention asks for the maintenance and continuous extension of a ledger of transactions. This “ledger” variation of the classical consensus problem has two fundamental objectives. *Consistency*: the ledgers in the views of any two participants are consistent with each other in the sense that they are either equal, or one is a prefix of the other. *Liveness*: there is a time bound, after which, any valid transaction will be incorporated in the view of the ledger of any honest participant. It is easy to see that these two properties individually can be easily satisfied by trivial protocols, however achieving them in tandem is an interesting protocol design question.

Ledger consensus is covered within the context of state machine replication (SMR) [Sch90]. More recently, the Bitcoin blockchain protocol [Nak08] provided a solution in a “permissionless” setting where participants do not know of each other and only a public setup operation is permitted. The protocol and its properties have been analyzed over a sequence of works [GKL15, PSs17, GKL17] that established its explicit guarantees regarding liveness and consistency.

The main task of a ledger consensus protocol is to continuously serialize the submitted transactions. While such serialization is not necessary for all applications of transaction services (e.g., payments is a notable exception [DM16, GKM⁺22]), serialization is of critical importance in the context of smart contracts. Indeed, when multiple parties interact with the same contract, the ordering of transactions can be crucial in terms of the effects that each transaction may have on the smart contract state. Observe that the properties of consistency and liveness necessitate that a certain serialization must take place (as the ledger must be well defined as a sequence of transactions) nevertheless do not impose a lot of constraints on that serialization except for the straightforward fact that transactions submitted very far apart from each other (in particular at least one full liveness property window) will be serialized in the order they were submitted.

This state of affairs leaves undetermined how transactions that are submitted more closely to each other should be serialized. This is by far not a theoretical consideration: by exploiting the behavior of Ethereum decentralized finance (DeFi) contracts it was shown how to deploy Miner Extractable Value (MEV) attacks [DGK⁺20] that exploit the ability of Ethereum miners to influence transaction serialization. By strategically placing suitable DeFi transactions in the serialization order of submitted transactions, miners gain additional profits (beyond what is provided in terms of transaction fees and newly minted coins associated with block production).

Motivated by mitigating such MEV attacks, a series of works attempted to enhance the ledger with a fair-order policy based on the maintainers’ local receiving time of transactions. Two main directions emerged in this line of “receiver order fairness” approach. On one hand, batch order fairness [KZGJ20, KDK22] (and later further improvements [CMSZ22, KLS24]) focus on defining valid transaction order based on their local *relative* order. On the other hand, by assuming a system-wide clock, timed order fairness [Kur20, ZSC⁺20] specifies a suitable order for a pair of transactions, if there is a point in time t such that all honest parties receive one transaction before t and the other transaction after t .

It is worth observing that looking at the receiving end of transaction dissemination, provides at best modest protection against front-running attacks as it pits the adversary in a network game against the honest parties: the adversary may still attempt to front-run when it becomes aware of a transaction by rushing the network delivery of its own transactions. Such attacks violate what is known as “input causality” a concept studied early by Reiter and Birman [RB94] with the aim to protect against an adversary that settles a transaction that depends in a meaningful way on transactions that are pending. This property has been later studied in the context of atomic broadcast by Cachin *et al.* [CKPS01] and recently in blockchain systems by Malkhi and

Szalachowski [MS23]. It can be seen that input causality complements receiver order fairness as it focuses on the sender side and tries to protect transactions in transit from being meaningfully front-run. Nevertheless input causality cannot entirely substitute receiver order considerations, since, even if input causality holds perfectly, one would still want to mandate that serialization adheres to a fairness criterion that attempts to serialize transactions in a way consistent with the timings that they were generated, disseminated and received by the ledger maintainers.¹

1.1 Our Results

Motivated by the above considerations we set out to model formally in the universal composition (UC) setting [Can01] the concept of order fair transaction serialization and realize it under plausible assumptions in a proof-of-work (PoW) setting.

Our starting point is the previous UC formalization of [BMTZ17] that focused on abstracting ledger consensus as an ideal functionality and showing how it can be realized in the PoW setting. One of the key features of this functionality is its ledger “extend policy” that allows the ideal world adversary to drive the progress of the ledger while being subject only to the liveness and “chain quality” properties [GKL15]. Note that no order fairness is provided by this functionality as the adversary is free to choose the order of pending transactions at will and even inject transactions adaptively, based on the transactions that honest parties have submitted for inclusion to the ledger.

Modeling. To capture comprehensively all fair ordering considerations, we adapt the ledger functionality of [BMTZ17] so that: (i) It only leaks to the adversary transaction identifiers that do not carry any information about the transactions themselves. The adversary has to commit first to an ordering and then subsequently the contents of the transactions are revealed so that the ledger can be assembled. (ii) It offers a parametric extend policy that constraints the adversary to follow an order that belongs to a class of admissible reorderings of the input transactions. Observe that this two-pronged approach combines input causality and receiver fair order considerations. This ideal functionality model results to a class of fair ledgers.

The most stringent extend policy one can imagine for serializing transactions is the exact order in which they were submitted to the ideal functionality. Note that in many ways such a “perfect” order is unreasonable; taking into account the global clock in the model, it could be the case that two transactions are “simultaneously” delivered w.r.t. the global clock. It follows that even though the perfect transaction order is available to the ideal functionality, it would be unreasonable to expect to realize it. Following this, our strictest formalization of sender order fairness is the one that restricts the adversary to conform to the arrival times to the ledger functionality as recorded by the global clock functionality. It is easy to see that one cannot expect to realize that level of fairness either. The reason is that in any setting where transaction delivery is not instantaneous (a setting that covers pretty much all reasonable deployment scenarios), there is no conceivable way that a ledger system can obey it. We formalize this result and in fact prove a much stronger *relativized* version of this impossibility theorem: even given any “private” helper functionality (i.e., one that is just responding to the caller) and a correlated random string, it is infeasible to achieve sender order fairness in a network with bounded delays.

The above results suggest that a relaxation is in order. To capture this, we introduce a natural but more relaxed sender order fairness property, δ -approximate order fairness, where the adversary is allowed to violate sender order fairness but only in a limited way, namely for transactions that are

¹To see why by an example, consider two DeFi traders that react to an external signal and perform a certain type of transaction (e.g., selling an asset). They are not aware of each other’s transaction so input causality is not a consideration; but it is still desirable that the system respects the order with which they submit their transactions.

sent δ time apart. It is easy to show that our impossibility result for sender order fairness extends to the setting where δ is below the network delay Δ , even in the relativized sense as described above. This leaves open the question of whether there exists a plausible helper functionality under which we can prove Δ -approximate order fairness.

It will not take long for the reader to guess a helper ideal functionality under which it is fairly easy to achieve our Δ -approximate sender order fairness formulation: For example, consider a functionality that encrypts all valid transactions and allows parties to disseminate them in an encrypted form so that their validity can be publicly verified and subsequently decrypt them only when they are settled. Given such functionality, it is possible for parties to disseminate transactions in an encrypted form and subsequently open them to reconstruct the state of the ledger. While this approach can work (and indeed it is folklore, see also [MS23] for a specific implementation) it imposes a rather heavy price if it is to be somehow realized from simpler components: it requires a shared private state (in order to facilitate decryption) and an ability to realize that the settlement of the encrypted transactions has taken place (since premature decryption of transactions would break security). Maintaining a shared private state can be expensive in the permissionless setting, e.g., using techniques such as YOSO MPC [BGG⁺20] it is possible for a committee to keep redistributing a secret-key of a threshold encryption function — with the downside of a quadratic communication complexity (in the security parameter) overhead at each round to facilitate the continuous re-sharing of the private key between successive committees.

Instead, poised to obtain a construction relevant to practice, we focus on the concept of a private functionality, thinking of it as a trusted execution enclave in the model of [PST17]. We ask whether it is possible to realize our approximate sender anonymous ledger functionality in a hybrid setting where the private functionality is instantiated as an enclave and is seeded only by an initialization string that is drawn from the correlated randomness functionality (this captures the ability of an enclave to be safely initialized by the enclave issuer). In particular this means that our enclave functionality does not have any shared private state across different enclave instances: each enclave has its own independent signing and encryption key. Finally, we also insist on transaction submission being a public operation that does not require access to an enclave.

Protocol overview. We first show how to enforce input-causality, and then we show how to lift the security of our protocol to realize a ledger that has Δ -approximate order fairness. Our protocol is built on top of a Nakamoto-style PoW blockchain $\mathcal{C}$, whose maintainers are equipped with a stateless² trusted execution environment [CD16] (enclave for brevity). Each enclave is equipped with its own public-key encryption key pair (pk, sk) ³, where only pk is revealed, while sk is hidden to everyone (including the owner of the enclave). Every output generated by the enclave is signed with a secret key known only to the enclave, but verifiable by anyone holding an *attested* verification key VK .

We require the maintainers to publish the enclave public-keys (pk, VK) on the blockchain⁴, upon registration. When a client wants to issue a transaction tx , it first samples a subset of polylogarithmic size S_{pk} of the registered public-keys (more detail on how S_{pk} is chosen is provided

²By stateless here we mean without secure counters, or without any mechanism that prevents the adversary from resetting the enclave.

³Note that in the hybrid model where communication takes place strictly via the diffusion functionality $\mathcal{F}_{\text{Diffuse}}$ that we use to model the peer-to-peer network, sharing a secret key sk among n enclaves requires diffusing n distinct messages from a single source — a prohibitive overhead. Contrary, in our protocol, in the worst case, each party has to diffuse polylogarithmic in the number of enclaves sized messages. Combining the capabilities of the enclave with the gossiping protocol over point-to-point channels may improve communication complexity — we leave exploring this interesting direction for future work.

⁴The maintainer will also issue the hash of the code that the enclave will run, which will make sure that every registered enclave will behave accordingly to what our protocol prescribes.

later) and encrypts $(\mathbf{tx}, h)$ using all the selected public keys, where h corresponds to the block-header of $\mathcal{C}$, thus obtaining a set of ciphertexts ct . Then, the client generates a zero-knowledge proof π proving that all the ciphertexts contain the same transaction and that this transaction is not equal to 0^κ . The client then diffuses to the network (ct, π) .

Each maintainer verifies the zero-knowledge proof, and if valid, it selects the ciphertext generated using its public key $\mathbf{pk}$ (if such a ciphertext does not exist, then (ct, π) is simply ignored). Then the maintainer queries the enclave with the ciphertext along with h' , which corresponds to the current blockchain header. Upon receiving the query, the enclave decrypts the ciphertext, thus obtaining the pair $(\mathbf{tx}, h)$ and checks how many blocks apart the chain with header h is compared to h' . If the number of blocks is more than Λ , then the enclave returns $\mathbf{tx}$ to the caller, else it waits to receive a new block header that satisfies the above condition.

This mechanism guarantees that an honest transaction can be decrypted only after a minimum amount of time, which is defined by the number of rounds required to add Λ blocks to the PoW blockchain. Moreover, the selection of $S_{\mathbf{pk}}$ guarantees that at least one public-key in the set belongs to an honest maintainer (more detail on this later), hence, the honest transactions will be surely decrypted. Once $\mathbf{tx}$ is obtained, it is diffused to the network, and it is the maintainers' goal now to include $\mathbf{tx}$ in the blockchain. Given that the underlying blockchain protocol we devise provides liveness, we are guaranteed that all honest transactions will appear in the blockchain.

The above approach seems to not cover the following attack. A malicious party could generate a ciphertext, and send it only to corrupted maintainers. The maintainers can decrypt the ciphertext as an honest party would, thus obtaining a signature for $\mathbf{tx}$, and decide whether to diffuse it or not in the network based on honest transactions that have been already decrypted and diffused in the network. This form of *adaptive rejection* is clearly incompatible with our goals.

To solve this problem we design a selection process for the set $S_{\mathbf{pk}}$, which guarantees that the party generating the set (even if it is corrupted) always includes in $S_{\mathbf{pk}}$ a public-key of an honest maintainer. In particular, we prove that as long as some constant fraction of the enclave owners is honest, then $S_{\mathbf{pk}}$ will contain with overwhelming probability at least one public-key of an honest party. We note that this could be achieved by simply requiring each transaction issuer to encrypt the transaction using all the enclave public keys. We instead propose an approach that yields the set $S_{\mathbf{pk}}$ being of polylogarithmic size. We refer the reader to the technical section for more details on this. On a final note, we stress that the enclaves we use are quite simple. In particular, no synchronization is required among them, and the adversary has full control of the internal clock of the enclaves it owns.

Transaction timestamping. We will now argue how to modify the blockchain $\mathcal{C}$, to finally realize our ledger with Δ -approximate order fairness (recall that Δ denotes the network delay). We modify the Nakamoto-style PoW $\mathcal{C}$ by binding the mining procedure of the blocks that form the chain $\mathcal{C}$ with a new type of blocks called *profile blocks*, using 2×1 (pronounced *two-for-one*) PoW [GKL15]. In 2×1 PoW, a hash function output u is checked twice for u and its reversed bit-string $[u]^R$. If $u < T$, a block that extends the chain $\mathcal{C}$ is produced; and if $[u]^R < T$, a new profile block (PB) is mined (which we will detail soon). In such a way, the mining procedure of the chain $\mathcal{C}$ and profile blocks are *bound* together and the adversary cannot gain advantage on either type of block by dropping from mining the other.

The blockchain $\mathcal{C}$ we use for our protocol is a sequence of blocks and all validation algorithms follow the longest-chain rule. The only difference between our chain $\mathcal{C}$ and that in, e.g., Bitcoin, is that blocks in $\mathcal{C}$ do not include transactions directly. The profile blocks instead are those storing the actual transactions, which in our case will correspond to a list of pairs of the form $(\mathbf{tag}, t)$ where $\mathbf{tag} = (ct, \pi)$ (which represents a *transaction identifier*) and t denotes the local receiving time. For a

transaction $\mathbf{tx}$ with identifier $\mathbf{tag}$ to be declared as *included* in a block $\mathcal{B}$ of the ledger, our protocol requires that the majority of the profile blocks connected to K consecutive blocks report $(\mathbf{tag}, \cdot)$. In a nutshell, we require each party generating a profile block to vote for the transaction and the position this transaction should finally have once it is included in the ledger.

In more detail, we employ two additional parameters for validating profile blocks — the recency parameter R , which is used to guarantee the freshness of profile blocks, and the profile window length parameter K . Specifically, a profile block PB should be allowed to be connected to a valid block $\mathcal{B}$ in the settled part of the blockchain, only if PB has not been mined too long ago. Let ℓ denote the height of block $\mathcal{B}$ that PB attaches to. PB is considered as a valid profile block only when PB is included in blocks with height less than $\ell + R$.

Finally, the timestamp of $\mathbf{tx}$ (or, its position in the final ledger) is computed by calculating the median timestamps of $\mathbf{tag}$ from all profile blocks included in blocks with height at most $\ell + K$. Note that if a profile block in the K -block window does not report an entry $(\mathbf{tag}, \cdot)$, it is counted as reporting $(\mathbf{tag}, +\infty)$ (i.e., the miner had never received $\mathbf{tx}$ thus cannot decrypt it). Formally, the timestamping of the transaction is computed as follows: $\text{TS}(\mathbf{tag}) \triangleq \text{med}\{t \mid (\mathbf{tag}, t) \in \text{PB} \in \mathcal{B} \wedge \ell \leq \text{height}(\mathcal{B}) < \ell + K\}$. The median provides Δ -order fairness because we can argue that in the K -block window associated with $\mathbf{tag}$ the majority of the timestamps are generated by honest parties.

On the enclave assumption. For our protocol we rely on enclaves, and one may wonder whether the use of enclaves trivializes the problem we are trying to solve. Indeed, it could be that realizing a ledger that achieves δ -order fairness with $\delta < \Delta$ becomes a trivial task with such assumptions. We prove that our result is optimal. In particular, we argue that even under stronger assumptions than the ones we use (e.g., enclaves synchronized with each other) it is impossible to design a protocol with better fairness than the one we provide. The high-level intuition behind this is that order fairness is strongly influenced by network delay. We also conjecture that it is possible to obtain a similarly optimal result by employing out of the box YOSO MPC [BGG⁺20] with the threshold encryption approach of [MS23] — nevertheless the relevance of such a construction is of less interest in practice and for this reason we do not pursue it here.

We also highlight that realizing sender order fairness and processing transactions in our protocol requires possession of a trusted enclave and hence it is only possible for permissioned nodes; specifically, in our setting such a “permissioned” node refers to a node that has been initialized with such an enclave and can be authenticated as such by other permissioned nodes. Note that our protocol is a PoW-based blockchain and hence it allows non-permissioned nodes to still contribute to the protocol without processing encrypted transactions; in fact such un-permissioned blocks may even contain transactions that are submitted unencrypted for which the clients do not wish to have them protected for sender-order fairness. As a side remark, this simple observation suggests that our protocol can be implemented as a “soft-fork” over Bitcoin — we leave the details of this implementation to be explored in future work.

Additionally, the security of our protocol relies on two assumptions — honest majority in terms of computational power and a constant fraction of the enclaves being possessed by honest hosts. While the second assumption is arguably weaker, we comment on how these two assumptions can be unified into one: Suppose all miners are equipped with enclaves and the honest parties account for the majority of computational power yet possess only a minority constant number of enclaves (i.e., the adversary is allowed to possess an arbitrary number of enclaves). Our approach is to extend the 2×1 PoW to 3×1 PoW (by checking non-overlapping 0s at different positions of the RO output), thus additionally mining the enclave public keys (each message mines only one key). These PoW-ed enclave public keys are included in the blockchain, and the key subset selection

procedure is then applied on the set of public keys with *weights* based on their PoW (for instance, a key with two unique PoW-ed messages has twice the weight of another key with one unique message). This adaption, with appropriate protocol parametrization, guarantees that the majority of weighted public keys are controlled by honest parties regardless of the fraction of honest keys.

Finally, one may consider it a compromise to use enclaves in a permissionless setting: we argue that this is not the case in practice. For instance, it is already the case that Bitcoin miners use specialized hardware to engage in mining and there are a handful of providers that offer such hardware in the market. Transitioning to a setting where such specialized hardware is also utilizing a trusted enclave may be a small step — especially if the benefits are substantial — as we demonstrate here: optimal sender order fairness.

2 Preliminaries

UC basics. We provide our protocols and security proofs in Canetti’s universal composition (UC) framework [Can00]. We discuss the main components of our real-world model (including the associated hybrids).

We assume that the reader is familiar with simulation-based security and has basic knowledge of the (G)UC framework. We review all the aspects of the execution model that are needed for our protocols and proof, but omit some of the low-level details and refer the interested reader to relevant works wherever appropriate.

We now recall the mechanics of activations in UC. In a UC protocol execution, an honest party (ITI) gets activated either by receiving an input from the environment, or by receiving a message from one of its hybrid functionalities (or from the adversary). Any activation results in the activated ITI performing some computation on its view of the protocol and its local state, and ends with either the party sending a message to some of its hybrid functionalities, sending an output to the environment, or not sending any message at all. In any of these cases, the party loses the activation.⁵ We denote the identities of parties by P_i , i.e. $P_i = (\text{pid}_i, \text{sid}_i)$, and call P_i a party for short. The index i is used to distinguish two identifiers, i.e., $P_i \neq P_j$, and otherwise carries no meaning. We will assume a central adversary $\mathcal{A}$ who gets to corrupt miners and might use them to attempt to break the protocol’s security. As is common in (G)UC, the resources available to the parties are described as hybrid functionalities. Our protocols are synchronous (G)UC protocols [BMTZ17]: parties have access to a (global) clock setup, denoted by $\mathcal{G}_{\text{Clock}}$ and can communicate over a network diffuse functionality $\mathcal{F}_{\text{Diffuse}}$ (mode details on these functionalities are provided later). We next sketch its main components: All functionalities, protocols, and setups have a dynamic party set. I.e., they all include special instructions allowing parties to register and deregister, and allow the adversary to learn the current set of registered parties. Additionally, global setups allow any other setup (or functionality) to register and deregister with them⁶ and also allow other setups to learn their set of registered parties.

Clock and diffusion functionality. Following the treatment in [BMTZ17], we model the synchronous processors and bounded-delay network as $\mathcal{G}_{\text{Clock}}$ and $\mathcal{F}_{\text{Diffuse}}$ respectively. The global clock $\mathcal{G}_{\text{Clock}}$ maintains a round index for each session, and this index can only be forwarded when all registered honest parties (in this session) have finished their computation for this round. The diffuse functionality $\mathcal{F}_{\text{Diffuse}}$ captures Δ -bounded network — i.e., when a message (either honest or

⁵In the latter case the activation goes to the environment by default.

⁶We note that the functionality can learn the code of the registering party (functionality) by considering the *extended identifier* in [BCH⁺20].

adversarial) is reached by at least one honest party at round r , it is guaranteed to be delivered to all honest parties before round $r + \Delta$. A detailed description of these functionalities is presented in Section A.

Global random oracles. The hash function H to generate PoW is modeled as a (global) random oracle $\mathcal{G}_{\text{RO}}$; by convention, we use a wrapper functionalities $\mathcal{W}(\mathcal{G}_{\text{RO}})$ to restrict the adversarial and *environmental* access to $\mathcal{G}_{\text{RO}}$. We assume honest majority in terms of the computational power to solve PoWs. I.e., in every round the RO wrapper allows more accesses from the honest parties than the corrupted (and environmental) ones. We also adopt a restricted programmable and observable global random oracle $\mathcal{G}_{\text{rpoRO}}$ [CDG⁺18] to model a hash function G that is different from the one used to generate PoW. We discuss these two global random oracles in detail in Section A.

Non-interactive zero knowledge. For our construction we use a non-interactive, straight-line simulation extractable (NISLE) proof system that relies on $\mathcal{G}_{\text{rpoRO}}$ as the only setup assumption. A protocol that satisfies such a security definition is provided in [LR22]. We assume familiarity with the notion of simulation extractable NIZK, and refer to Section A for the formal definition.

Pseudorandom functions, one-way functions and hardcore bits. Let $F : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ be a keyed function. F is a pseudorandom function (PRF) if for all PPT distinguisher D it holds that $|\Pr_{k \leftarrow \{0, 1\}^n}[D^{F_k(\cdot)} = 1] - \Pr_{f \leftarrow \mathcal{F}_n}[D^f(\cdot) = 1]| \leq \text{negl}(n)$. A function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is a one-way function if there exists an efficient algorithm for evaluating f , and for all PPT adversary $\mathcal{A}$, we have $\Pr_{x \leftarrow \{0, 1\}^n}[\mathcal{A}(f(x)) \in f^{-1}(f(x))] \leq \text{negl}(n)$. A hardcore bit is a function $b : \{0, 1\}^* \rightarrow \{0, 1\}$ such that for all PPT adversary $\mathcal{A}$, it holds that $\Pr_{x \leftarrow \{0, 1\}^n}[\mathcal{A}(f(x)) = b(x)] \leq \frac{1}{2} + \text{negl}(n)$.

3 Ledger Functionality with Parametric Extend Policy

In order to capture fairness in transaction serialization, we extend the ledger functionality $\mathcal{G}_{\text{Ledger}}$ in [BMTZ17] by enhancing the extend policy with parametric order policy and revising the transaction information leakage behavior (which we denote by $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$). We highlight our main adaptations in this section.

Parametric extend policy. The ledger in [BMTZ17] is parametrized with an `ExtendPolicy` algorithm such that, for every new block, the adversary is allowed to propose an arbitrary sequence using an arbitrary subset of the transaction buffer, as long as (i) all valid transactions are included timely (liveness); (ii) the proposed ledger grows at a relatively steady rate (chain growth); and (iii) the fraction of honest blocks in any section of the blockchain is lower-bounded (chain quality). This captures exactly the transaction inclusion mechanism in real-world Bitcoin. Meanwhile, subject to the designated chain quality parameter, front-running is possible during the ongoing of a sequence of blocks that is at most the same number of the common prefix parameter.

In order to capture order fairness and towards a more modular design of the core ledger extending mechanism, we propose a new `ExtendPolicy` in Algorithm 1 which (i) can be parameterized with different fair-order policies; and (ii) revises the liveness guarantee check (which is necessary since we decouple the consensus and transaction settlement). To improve accessibility, we mark in blue the differences compared with the original `ExtendPolicy` in [BMTZ17].

Upon receiving a proposed block from the adversary, `ExtendPolicy` iterates over all transactions; for each transaction `tx`, its *order* and validity with respect to the current state and buffer is evaluated. If `tx` turns out to be a valid transaction and extends the ledger following the fair-order policy, the algorithm goes for the next transaction; otherwise, it aborts and returns default blocks from `DefaultExtension`. The transaction validation procedure follows that in [BMTZ17]. Regarding the fair-order examination, we let `ValidOrder` and `DefaultExtension` be parameters of Algorithm 1.

ValidOrder takes the target transaction **tx**, current ledger state **state** and pending transaction buffer **buffer** as input and returns whether **tx** is a valid extension with respect to **state** and **buffer** under a designated fair-order policy; **DefaultExtension** takes the honest input sequence $\vec{\mathcal{I}}_H^T$, the current ledger state **state** and pending transaction buffer **buffer** and returns the default block under the designated fair-order policy that ledger $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ should specify. The **ValidOrder** algorithm is called for every transaction (Line 17 in Algorithm 1). Different **ValidOrder** and **DefaultExtension** algorithms can implement different fair-order policies, and we discuss them in Section 4.

Alongside with the new **ValidOrder** examination, we revise the transaction liveness check. Recall that in [BMTZ17], once an honest block is proposed, it should include all relatively recent pending transactions in the buffer to avoid the violation of liveness. Looking ahead, our protocol employs a blockchain scheme that decouples consensus and transaction inclusion (a similar framework has been used in, e.g., [PS17, FGKR20]), hence the liveness parameter is not exactly the same as **windowSize** (which indicates the number of blocks that needs to be pruned to achieve a consistent view on the level of blockchains). We adopt a new parameter **waitTime** for transaction liveness, so the old transaction checks only those sent before time $\tau_{\mathcal{L}} - \text{waitTime}$. Additionally, old transactions will be excluded if they commit to an invalid order with respect to the current ledger. See Line 23 and 24 in Algorithm 1.

Algorithm ExtendPolicy($\vec{\mathcal{I}}_H^T, \text{state}, \text{NxtBC}, \text{buffer}, \vec{\tau}_{\text{state}}$)

```

1:  $\vec{N}_{\text{df}} \leftarrow \text{DefaultExtension}(\vec{\mathcal{I}}_H^T, \text{state}, \text{NxtBC}, \text{buffer}, \vec{\tau}_{\text{state}})$ 
2:  $\tau_{\mathcal{L}} \leftarrow$  current ledger time (computed from  $\vec{\mathcal{I}}_H^T$ )
3: Create local copies of the values buffer, state and  $\vec{\tau}_{\text{state}}$ .
4: Parse NxtBC as a vector  $((\text{hFlag}_1, \text{NxtBC}_1), \dots, (\text{hFlag}_n, \text{NxtBC}_n))$ 
5:  $\vec{N} \leftarrow \varepsilon$ 
6: if  $|\text{state}| \geq \text{windowSize}$  then  $\tau_{\text{low}} \leftarrow \vec{\tau}_{\text{state}}[|\text{state}| - \text{windowSize} + 1]$  else  $\tau_{\text{low}} \leftarrow 0$ 
7: oldValidTxMissing  $\leftarrow$  false
8: for each list NxtBCi of transaction IDs do
9:   Use the txid contained in NxtBCi to determine the list of transactions
10:  Let  $\vec{\text{tx}} = (\text{tx}_1, \dots, \text{tx}_{|\text{NxtBC}_i|})$  denote the transactions of NxtBCi
11:  if tx1 is not a coinbase transaction then
12:    return  $\vec{N}_{\text{df}}$ 
13:  else
14:     $\vec{N}_i \leftarrow \text{tx}_1$ 
15:    for  $j = 2$  to  $|\text{NxtBC}_i|$  do
16:       $\text{st}_i \leftarrow \text{Blockify}(\vec{N}_i)$ 
17:       $\triangleright$  Default Extension if proposal violates fair order
18:      if ValidOrder(txj, state  $\parallel$  sti, buffer) = false then return  $\vec{N}_{\text{df}}$ 
19:       $\triangleright$  Default Extension if proposal includes invalid transaction
20:      if ValidTX(txj, state  $\parallel$  sti) = false then return  $\vec{N}_{\text{df}}$ 
21:       $\vec{N}_i \leftarrow \vec{N}_i \parallel \text{tx}_j$ 
22:    end for
23:     $\text{st}_i \leftarrow \text{Blockify}(\vec{N}_i)$ 
24:  end if
25:   $\triangleright$  Test that all old transactions are included.
26:  for each BTX = (tx, txid,  $\tau'$ , P)  $\in$  buffer s.t. P is honest and  $\tau' < \tau_{\mathcal{L}} - \text{waitTime}$  do
27:    if ValidTX(tx, state  $\parallel$  sti) = true and ValidOrder(tx, state  $\parallel$  sti, buffer) = true and
28:    tx  $\notin \vec{N}_i$  then
29:      oldValidTxMissing  $\leftarrow$  true

```

```

26:     end if
27:   end for
28:    $\vec{N} \leftarrow \vec{N} \parallel \vec{N}_i$ 
29:    $\mathbf{state} \leftarrow \mathbf{state} \parallel \mathbf{st}_i$ 
30:    $\vec{\tau}_{\mathbf{state}} \leftarrow \vec{\tau}_{\mathbf{state}} \parallel \tau_{\mathcal{L}}$ 
  ▷ Must not proceed with too many adversarial blocks
31:    $j \leftarrow \max\{\{\mathbf{windowSize}\} \cup \{k \mid \mathbf{st}_k \in \mathbf{state} \wedge \text{proposal of } \mathbf{st}_k \text{ had hFlag} = 1\}\}$ 
32:   if  $|\mathbf{state}| - j \geq \eta$  then return  $\vec{N}_{\text{df}}$ 
33:   if  $|\mathbf{state}| \geq \mathbf{windowSize}$  then
34:      $\tau_{\text{low}} \leftarrow \vec{\tau}_{\mathbf{state}}[|\mathbf{state}| - \mathbf{windowSize} + 1]$ 
35:   else
36:      $\tau_{\text{low}} \leftarrow 0$ 
37:   end if
38: end for
  ▷ A sequence of blocks cannot take too much time
39: if  $\tau_{\text{low}} > 0$  and  $\tau_{\mathcal{L}} - \tau_{\text{low}} > \mathbf{maxTime}_{\text{window}}$  then return  $\vec{N}_{\text{df}}$ 
  ▷ Bootstrapping cannot take too much time
40: if  $\tau_{\text{low}} = 0$  and  $\tau_{\mathcal{L}} - \tau_{\text{low}} > 2 \cdot \mathbf{maxTime}_{\text{window}}$  then return  $\vec{N}_{\text{df}}$ 
41: if oldValidTxMissing then return  $\vec{N}_{\text{df}}$     ◁ Old enough, valid transactions should be included
42: return  $\vec{N}$ 

```

Algorithm 1: *The extend policy employed by the fair ledger functionality $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$.*

Transaction leakage after settlement. Recall that while Bitcoin is pseudonymous, all transactions are actually public and transparent (both on-chain and in the mempool). The ledger functionality in [BMTZ17] thus leaks all information to the adversary once it receives a transaction. This gives the adversary the full power to adaptively issue transactions, even within the same round as the victim ones. In order to capture the goal that the adversary cannot finalize a transaction that depends in a meaningful way of any pending transactions (i.e., input causality, cf. [CKPS01]), the ideal functionality must hide the transaction content and metadata before it gets settled.

We make the following two enhancements to $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$. On one hand, upon a transaction is sent to the functionality, $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ book-keeps the transaction as $(\mathbf{tx}, \text{txid}, \tau_{\mathcal{L}}, \mathbf{P})$ where $\mathbf{tx}$ denotes the plain transaction (i.e., content and metadata including issuer’s signature), txid is a unique random tag generated upon receipt, $\tau_{\mathcal{L}}$ is the time that this transaction is sent and $\mathbf{P}$ denotes the sender. Our new functionality $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ immediately leaks transaction length $|\mathbf{tx}|$, identifier txid , receiving time $\tau_{\mathcal{L}}$ and sender $\mathbf{P}$ to the adversary (recall that we assume a network adversary); however, $\mathbf{tx}$ itself remains hidden (note that in [BMTZ17] the entire tuple is leaked immediately). On the other hand, upon the adversary requests to read all transactions in the buffer, $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ returns (i) the current state $\mathbf{state}$ where all transactions are transparent (indicating that they have been settled in the immutable ledger); and, (ii) transaction buffer $\mathbf{buffer}$ except that for each transaction tuple $(\mathbf{tx}, \cdot, \cdot, \cdot)$ in $\mathbf{buffer}$, $\mathbf{tx}$ is replaced with its length $|\mathbf{tx}|$ in the response. Precisely, upon receiving the READ command, the functionality returns $(\widehat{\mathbf{state}}, \widehat{\mathbf{buffer}})$ such that

$$\widehat{\mathbf{buffer}} \triangleq \{(|\mathbf{tx}|, \text{txid}, \tau, \mathbf{P}) \mid \text{BTX} = (\mathbf{tx}, \text{txid}, \tau, \mathbf{P}) \in \mathbf{buffer}\}. \quad (1)$$

In such a way, the ideal world adversary is limited to learn only the “existence” of a transaction $\mathbf{tx}$ before its settlement. To learn the full information of $\mathbf{tx}$, the adversary has to either propose

$\mathbf{tx}$ in the next block or wait for the default extension policy to handle $\mathbf{tx}$. In other words, the time interval that $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ hides the plain transaction is upper-bounded by liveness; but it is up to the ideal world adversary to decide whether to propose $\mathbf{tx}$ earlier and learn its content, or wait for the default extension. (Arguably, a good simulator never uses the default extension to learn $\mathbf{tx}$; details see the protocol analysis.) Also, notice that the sender P leaks information at the network level and is not necessarily linked with the transaction issuer. Thus $\mathbf{tx}$ remains completely secret before being committed to the ledger state.

We provide a complete description of our ledger $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$. For brevity, major revisions compared with [BMTZ17] are marked with [blue](#) text.

Functionality $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$

General: The functionality is parametrized by four algorithms `Validate`, `ExtendPolicy`, `Blockify`, and `predict-time`, along with two parameters `windowSize`, `Delay`, [waitTime](#) $\in \mathbb{N}$. The functionality manages variables `state`, `NxtBC`, `buffer`, $\tau_{\mathcal{L}}$ and $\vec{\tau}_{\text{state}}$. Initially, `state` $\leftarrow \varepsilon$, `NxtBC` $\leftarrow \varepsilon$, $\vec{\tau}_{\text{state}}$ $\leftarrow \varepsilon$, `buffer` $\leftarrow \emptyset$, $\tau_{\mathcal{L}}$ $\leftarrow 0$.

For each party $P_i \in \mathcal{P}$ the functionality maintains a pointer $\mathbf{pt}_i$ (initially set to 1) and a current-state view `statei` $\leftarrow \varepsilon$ (initially set to empty). The functionality keeps track of the timed honest-input sequence $\vec{\mathcal{I}}_H^T$ (initially $\vec{\mathcal{I}}_H^T \leftarrow \varepsilon$).

Party management: The functionality maintains the set of registered parties $\mathcal{P}$, the (sub-)set of honest parties $\mathcal{H} \subseteq \mathcal{P}$, and the (sub-set) of de-synchronized honest parties $\mathcal{P}_{DS} \subset \mathcal{H}$. The sets $\mathcal{P}$, $\mathcal{H}$, $\mathcal{P}_{DS}$ are all initially set to $\emptyset$. When a new honest party is registered at the ledger, if it is registered with the clock already then it is added to the party sets $\mathcal{H}$ and $\mathcal{P}$ and the current time of registration is also recorded; if the current time is $\tau_{\mathcal{L}} > 0$, it is also added to $\mathcal{P}_{DS}$. Similarly, when a party is deregistered, it is removed from both $\mathcal{P}$ (and therefore also from $\mathcal{P}_{DS}$ or $\mathcal{H}$). The ledger maintains the invariant that it is registered (as a functionality) to the clock whenever $\mathcal{H} \neq \emptyset$. A party is considered fully registered if it is registered with the ledger and the clock.

Upon receiving any input I from any party or from the adversary, send `(CLOCK-READ, sidC)` to $\mathcal{G}_{\text{Clock}}$ and upon receiving response `(CLOCK-READ, sidC,  $\tau$ )` set $\tau_{\mathcal{L}} \leftarrow \tau$, and do the following:

1. Let $\hat{\mathcal{P}} \subseteq \mathcal{P}_{DS}$ denote the set of de-synchronized honest parties that have been registered (continuously) since time $\tau' < \tau_{\mathcal{L}} - \text{Delay}$ (to both ledger and clock). Set $\mathcal{P}_{DS} \leftarrow \mathcal{P}_{DS} \setminus \hat{\mathcal{P}}$. On the other hand, for any synchronized party $P \in \mathcal{H} \setminus \mathcal{P}_{DS}$, if P is not registered to the clock, then $\mathcal{P}_{DS} \cup \{P\}$.
2. If I was received from an honest party $P_i \in \mathcal{P}$:
 - (a) Set $\vec{\mathcal{I}}_H^T \leftarrow \vec{\mathcal{I}}_H^T \parallel (I, P_i, \tau_{\mathcal{L}})$;
 - (b) Compute $\vec{N} = (\vec{N}_1, \dots, \vec{N}_{\ell}) \leftarrow \text{ExtendPolicy}(\vec{\mathcal{I}}_H^T, \text{state}, \text{NxtBC}, \text{buffer}, \vec{\tau}_{\text{state}})$ and if $\vec{N} \neq \varepsilon$ set `state` $\leftarrow \text{state} \parallel \text{Blockify}(\vec{N}_1) \parallel \dots \parallel \text{Blockify}(\vec{N}_{\ell})$ and $\vec{\tau}_{\text{state}} \leftarrow \vec{\tau}_{\text{state}} \parallel \tau_{\mathcal{L}}^{\ell}$ where $\tau_{\mathcal{L}}^{\ell} = \tau_{\mathcal{L}} \parallel \dots \parallel \tau_{\mathcal{L}}$;
 - (c) For each `BTX` $\in$ `buffer`: if `Validate(BTX, state, buffer)` = 0 then delete `BTX` from `buffer`. Also, reset `NxtBC` $\leftarrow \varepsilon$.
 - (d) If there exists $P_j \in \mathcal{H} \setminus \mathcal{P}_{DS}$ such that $|\text{state}| - \mathbf{pt}_j > \text{windowSize}$ or $\mathbf{pt}_j < |\text{state}_j|$, then set $\mathbf{pt}_k \leftarrow |\text{state}|$ for all $P_k \in \mathcal{H} \setminus \mathcal{P}_{DS}$.
3. Depending on the input I and the ID of the sender, execute the respective code:
 - *Submitting a transaction:* If $I = (\text{SUBMIT}, \text{sid}, \mathbf{tx})$ and is received from a party $P \in \mathcal{P}$ or from $\mathcal{A}$ (on behalf of a corrupted party P) do the following:
 - (a) Choose a unique transaction ID `txid` and set `BTX` $\leftarrow (\mathbf{tx}, \text{txid}, \tau_{\mathcal{L}}, P)$;
 - (b) If `Validate(BTX, state, buffer)` = true, then `buffer` $\leftarrow \text{buffer} \cup \{\text{BTX}\}$;
 - (c) [Send \(SUBMIT, \(BTX.tx, BTX.txid, BTX. \$\tau_{\mathcal{L}}\$, BTX.P\)\) to \$\mathcal{A}\$.](#)
 - *Reading the state:* If $I = (\text{READ}, \text{sid})$ is received from a fully registered party $P_i \in \mathcal{P}$ then set `statei` $\leftarrow \text{state}|_{\min\{\mathbf{pt}_i, |\text{state}|\}}$ and return `(READ, sid, statei)` to the requestor. If the requestor

is $\mathcal{A}$ then send $(\text{state}, \widehat{\text{buffer}}, \tilde{\mathcal{I}}_H^T)$ to $\mathcal{A}$ where $\widehat{\text{buffer}} \triangleq \{(|\text{tx}|, \cdot, \cdot, \cdot) \mid \text{BTX} = (\text{tx}, \cdot, \cdot, \cdot) \in \text{buffer}\}$ (cf. Equation (1)).

- *Maintaining the ledger state:* If $I = (\text{MAINTAIN-LEDGER}, \text{sid}, \text{minerID})$ is received by an honest party $P_i \in \mathcal{P}$ and (after updating $\tilde{\mathcal{I}}_H^T$ as above) $\text{predict-time}(\tilde{\mathcal{I}}_H^T) = \hat{\tau} > \tau_{\mathcal{L}}$ then send $(\text{CLOCK-UPDATE}, \text{sid}_C)$ to $\mathcal{G}_{\text{Clock}}$. Else send I to $\mathcal{A}$.
- *The adversary proposing the next block:* If $I = (\text{NEXT-BLOCK}, \text{hFlag}, (\text{txid}_1, \dots, \text{txid}_\ell))$ is sent from the adversary, update NxtBC as follows:
 - (a) Set $\text{listOfTxid} \leftarrow \varepsilon$;
 - (b) For $i = 1, \dots, \ell$ do: if there exists $\text{BTX} = (\cdot, \text{txid}, \tau_{\mathcal{L}}, P_i) \in \text{buffer}$ with ID $\text{txid} = \text{txid}_i$ then set $\text{listOfTxid} \leftarrow \text{listOfTxid} \parallel \text{txid}_i$;
 - (c) Finally, set $\text{NxtBC} \leftarrow \text{NxtBC} \parallel (\text{hFlag}, \text{listOfTxid})$ and output $(\text{NEXT-BLOCK}, \text{ok})$ to $\mathcal{A}$.
- *The adversary setting state-slackness:* If $I = (\text{SET-SLACK}, (P_{i_1}, \hat{\text{pt}}_{i_1}), \dots, (P_{i_\ell}, \hat{\text{pt}}_{i_\ell}))$, with $\{P_{i_1}, \dots, P_{i_\ell}\} \subseteq \mathcal{H} \setminus \mathcal{P}_{DS}$ is received from the adversary $\mathcal{A}$ do the following:
 - (a) If for all $j \in [\ell] : |\text{state}| - \hat{\text{pt}}_{i_j} \leq 4\text{windowSize}$ and $\hat{\text{pt}}_{i_j} \geq |\text{state}_{i_j}|$, set $\text{pt}_{i_j} \leftarrow \hat{\text{pt}}_{i_j}$ for every $j \in [\ell]$ and return $(\text{SET-SLACK}, \text{ok})$ to $\mathcal{A}$;
 - (b) Otherwise set $\text{pt}_{i_j} \leftarrow |\text{state}|$ for all $j \in [\ell]$.
- *The adversary setting the state for de-synchronized parties:* If $I = (\text{DESYNC-STATE}, (P_{i_1}, \text{state}'_{i_1}), \dots, (P_{i_\ell}, \text{state}'_{i_\ell}))$, with $\{P_{i_1}, \dots, P_{i_\ell}\} \subseteq \mathcal{P}_{DS}$ is received from the adversary $\mathcal{A}$, set $\text{state}_{i_j} \leftarrow \text{state}'_{i_j}$ for each $j \in [\ell]$ and return $(\text{DESYNC-STATE}, \text{ok})$ to $\mathcal{A}$.

Functionality 1: *The ledger functionality with parametric extend policy.*

4 Extend Policies with Sender Order Fairness

The parametric `ExtendPolicy` introduced in Section 3 allows us to insert a customized fair-order policy into $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ such that its output ledger state should follow a legitimate transaction sequence under the designated policy. In this section we consider two policy instances⁷. The first one is the most natural sender-side order fairness `SenderOrder` where transactions should be serialized by the order they are received by the ledger $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$. The second policy is a (necessary) relaxation on sender order fairness (namely, δ -`ApproxSenderOrder`) which asks for the order of two transactions if they reach $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ at times that are δ apart from each other. We discuss impossibility results with respect to both policies in the same setting as Bitcoin (i.e., with global clock and Δ -bounded delay diffusion network). Further, we show that the unfairness stemmed from network delay is impossible to circumvent — we prove that even if we employ additional powerful setups, still no δ -`ApproxSenderOrder` can be achieved for any $\delta < \Delta$.

4.1 Sender Order Fairness

Sender-side order fairness captures the natural desire that a ledger $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ serializes transactions based on the time that they are released by their senders. This order is well-defined by looking at the $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ transaction buffer and order transactions based on their arrival time recorded. Hence, every time the adversary tries to propose a transaction tx that is sent at time τ (i.e., $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ receives tx and records $(\text{tx}, \text{txid}, \tau, P)$), our `ValidOrder` algorithm in `SenderOrder` checks whether there exists

⁷Capturing receiver order fairness definitions, e.g., [KZGJ20, CMSZ22], can be an interesting direction for future work.

valid transactions in **buffer** with time $\tau' < \tau$. If such transaction exists, the proposed order is invalid and this will force the **ExtendPolicy** to return a default extension.

Regarding the **DefaultExtension** mechanism, since the original construction in [BMTZ17] has been proposing a default order that follows the one stored in the ledger, we follow that construction with one minor modification. Precisely, in [BMTZ17] a default extension will blockify all pending transactions in the buffer, while ours only include those that are about to violate liveness — again this adaption follows our blockchain scheme that decouples the consensus and transaction inclusion. We note that this adaption is insignificant, as a good simulator shall never allow the ledger to trigger this subroutine.

Algorithm SenderOrder

```

ValidOrder(tx, state, buffer)
1: Parse tx as (tx, txid,  $\tau$ , P)
   ▷ Extract the most recent transaction time in the ledger state.
2:  $\tau^* \leftarrow \max\{\tau' \mid \text{BTX} = (\text{tx}, \cdot, \tau', \cdot) \wedge \text{tx} \in \text{state}\}$ 
3:  $\vec{\text{tx}} \leftarrow \{\text{tx} \mid \text{BTX} = (\text{tx}, \cdot, \tau', \cdot) \in \text{buffer} \wedge \text{BTX} \notin \vec{N} \wedge \tau^* \leq \tau' < \tau\}$ 
4: for tx  $\in \vec{\text{tx}}$  do
5:   if ValidTX(tx, state) = true then return false
6: end for
7: return true

DefaultExtension( $\vec{\mathcal{I}}_H^T$ , state, NextBC, buffer,  $\vec{\tau}_{\text{state}}$ )
8:  $\tau_{\mathcal{L}} \leftarrow$  current ledger time (computed from  $\vec{\mathcal{I}}_H^T$ )
9:  $\vec{N}_{\text{df}} \leftarrow \text{tx}_{\text{minerID}}^{\text{base-tx}}$  of an honest miner
10: Let  $\vec{\text{tx}} = \{\text{tx} \mid \text{BTX} = (\text{tx}, \text{txid}, \tau'_{\mathcal{L}}, \text{P}) \in \text{buffer} \wedge \tau'_{\mathcal{L}} < \tau_{\mathcal{L}} - \text{waitTime}\}$ 
11: Sort  $\vec{\text{tx}}$  according to timestamps
12:  $\text{st} \leftarrow \text{Blockify}(\vec{N}_{\text{df}})$ 
13: repeat
14:   Let  $\vec{\text{tx}} = (\text{tx}_1, \dots, \text{tx}_{\ell})$  be the current set of (remaining) transactions
15:   for  $i = 1$  to  $\ell$  do
16:     if ValidTX( $\text{tx}_i$ , state || st) = true and ValidOrder( $\text{tx}_i$ , state || st, buffer) = true then
17:        $\vec{N}_{\text{df}} \leftarrow \vec{N}_{\text{df}} \parallel \text{tx}_i$ 
18:       Remove  $\text{tx}_i$  from  $\vec{\text{tx}}$ 
19:        $\text{st} \leftarrow \text{Blockify}(\vec{N})$ 
20:     end if
21:   end for
22: until  $\vec{N}_{\text{df}}$  does not increase any more
23: if  $|\text{state}| + 1 \geq \text{windowSize}$  then  $\tau_{\text{low}} \leftarrow \vec{\tau}_{\text{state}}[|\text{state}| - \text{windowSize} + 2]$  else set  $\tau_{\text{low}} \leftarrow 0$ 
24:  $c \leftarrow 1$ 
25: while  $\tau_{\mathcal{L}} - \tau_{\text{low}} > \text{maxTime}_{\text{window}}$  do
26:   Set  $\vec{N}_c \leftarrow \text{tx}_{\text{minerID}}^{\text{base-tx}}$  of an honest miner
27:    $\vec{N}_{\text{df}} \leftarrow \vec{N}_{\text{df}} \parallel \vec{N}_c$ 
28:    $c \leftarrow c + 1$ 
29:   if  $|\text{state}| + c \geq \text{windowSize}$  then  $\tau_{\text{low}} \leftarrow \vec{\tau}_{\text{state}}[|\text{state}| - \text{windowSize} + c + 1]$  else  $\tau_{\text{low}} \leftarrow 1$ 
30: end while
31: return  $\vec{N}_{\text{df}}$ 

```

Algorithm 2: The *ValidOrder* predicate and *DefaultExtension* algorithm for sender fairness.

The sender fairness **SenderOrder** is our desideratum; unfortunately, **SenderOrder** is impossible

to realize, even in the synchronous network (i.e., $\Delta = 1$ in $\mathcal{F}_{\text{Diffuse}}^\Delta$). The high level intuition is that transactions issued by corrupted parties can always reach honest parties in two consecutive rounds by sending to one honest party P at round r and let P diffuse it to others at round $r + 1$, where in the case of honest party issuing transactions, the sender can book-keep their receiving time to the next round. The fact that the adversary $\mathcal{A}$ can exploit this one round advantage allows him to make the following two executions identical: in the first execution, $\mathcal{A}$ sends a transaction $\mathbf{tx}$ at round r to only one honest party and in the second execution he sends $\mathbf{tx}$ to all honest parties at round $r + 1$. Later, in the second execution $\mathcal{A}$ lets one corrupted party behave like it receive and diffuses the message at round r . In the first execution, $\mathbf{tx}$ in $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ owns timestamp r while in the second execution it owns timestamp $r + 1$, however these two executions are indistinguishable.

4.2 Approximate Sender Order Fairness

Given that **SenderOrder** is unachievable unless under unrealistic network assumptions (that is, the delivery of a transaction is instant and before any transaction that is issued later thus every honest parties see the same sequence of transactions), we next propose a natural alternative that relaxes the sender order fairness, giving up on asking for the order of two transactions if they enter the network at approximately the same time.

We leave the **DefaultExtension** procedure the same as that in **SenderOrder** where an order following what $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ receives is proposed (hence we omit it in the algorithm description). Note that such default extension will still yield a legitimate order with respect to δ -**ApproxSenderOrder**. Regarding the **ValidOrder** algorithm, when the adversary is to propose a transaction $\mathbf{tx}$ sending the ledger at time τ , all transactions in the buffer which are sent at time before $\tau - \delta$ and are valid with respect to state should be proposed before $\mathbf{tx}$; otherwise, **ValidOrder** returns false to instruct the ledger to use the **DefaultExtension**.

Algorithm δ -ApproxSenderOrder

```

ValidOrder( $\mathbf{tx}, \vec{N}, \text{buffer}$ )
1: Parse  $\mathbf{tx}$  as  $(\mathbf{tx}, \text{txid}, \tau, P)$ 
   ▷ Extract the most recent transaction time in the ledger state.
2:  $\tau^* \leftarrow \max\{\tau' \mid \text{BTX} = (\mathbf{tx}, \cdot, \tau', \cdot) \wedge \mathbf{tx} \in \text{state}\}$ 
3:  $\vec{\mathbf{tx}} \leftarrow \{\text{BTX} = (\cdot, \cdot, \tau', \cdot) \mid \text{BTX} \in \text{buffer} \wedge \text{BTX} \notin \vec{N} \wedge \tau^* \leq \tau' < \tau - \delta\}$ 
4: for  $\mathbf{tx} \in \vec{\mathbf{tx}}$  do
5:   if ValidTX( $\mathbf{tx}, \text{state}$ ) = true then return false
6: end for
7: return true

```

Algorithm 3: The *ValidOrder* predicate for approximate sender fairness.

Impossibility of δ -ApproxSenderOrder with Δ -bounded network delay for $\delta < \Delta$. We show that in the same setting as Bitcoin (i.e., with $\mathcal{G}_{\text{Clock}}$, $\mathcal{W}(\mathcal{G}_{\text{RO}})$ and $\mathcal{F}_{\text{Diffuse}}^\Delta$), it is impossible to guarantee fairness for two transactions $\mathbf{tx}, \mathbf{tx}'$ that enters the system less than δ rounds apart from each other for any $\delta < \Delta$.

Theorem 1. *In the $(\mathcal{G}_{\text{Clock}}, \mathcal{W}(\mathcal{G}_{\text{RO}}), \mathcal{F}_{\text{Diffuse}}^\Delta)$ -hybrid environment, there exist no protocol Π that securely realizes $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ parameterized with δ -ApproxSenderOrder for any $\delta < \Delta$.*

Proof. Suppose towards a contradiction, there exists a protocol Π that securely realizes $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ parameterized with δ -ApproxSenderOrder in the $(\mathcal{G}_{\text{Clock}}, \mathcal{W}(\mathcal{G}_{\text{RO}}), \mathcal{F}_{\text{Diffuse}}^\Delta)$ -hybrid environment.

Consider two transactions $\mathbf{tx}, \mathbf{tx}'$. Let τ, τ' denote the timestamp that $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ assigns to them in the transaction buffer respectively. Consider a world W_1 where $\mathbf{tx}$ is sent more than δ yet less than Δ rounds before $\mathbf{tx}'$ (i.e., $\tau + \delta < \tau' < \tau + \Delta$). I.e., δ -ApproxSenderOrder would force $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ to output $\mathbf{tx} \prec \mathbf{tx}'$. Regarding the transaction diffusion pattern in W_1 , assume all but the sender of $\mathbf{tx}$ receives $\mathbf{tx}$ at time $\tilde{\tau}$ such that $\tau' < \tilde{\tau} < \tau + \Delta$; and all but the sender of $\mathbf{tx}'$ receives $\mathbf{tx}'$ at time $\tilde{\tau}'$ such that $\tau' \leq \tilde{\tau}' < \tilde{\tau}$. I.e., for all parties except the senders, they saw $\mathbf{tx}' \prec \mathbf{tx}$ unanimously. Then, we consider the following three scenarios.

First, neither the sender of $\mathbf{tx}$ nor the sender of $\mathbf{tx}'$ has the chance to send a message. We prove that for all other protocol participants in W_1 this is indistinguishable from the world W_2 where $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ shall output $\mathbf{tx}' \prec \mathbf{tx}$. Consider the following scenario in W_2 : two transactions $\mathbf{tx}, \mathbf{tx}'$ are sent at time τ', τ respectively and are delivered to all other parties at time $\tilde{\tau}, \tilde{\tau}'$ respectively. Due to δ -ApproxSenderOrder, $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ outputs $\mathbf{tx}' \prec \mathbf{tx}$. The local views of all other parties are identical to the ones in W_1 hence the messages they send are also identical.

Second, at least one of the two senders of $\mathbf{tx}, \mathbf{tx}'$ successfully sends at least one message. Note that this implies Π is a protocol such that the pairwise order of any two transactions could be dominated by a single party (or, a tiny fraction of resources to run the protocol). Again we show that W_1 is indistinguishable from the world W_3 where $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ shall output $\mathbf{tx}' \prec \mathbf{tx}$. Suppose, with out loss of generality, $\mathbf{tx}$ is generated by a corrupted party. Consider the following scenario in W_3 : the transaction timestamp in $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ and diffusion pattern are exactly the same as those in W_2 except that now the corrupted issuer of $\mathbf{tx}$ sends a message claiming that $\mathbf{tx}$ is sent at time τ (this is a valid message even if Δ is known and parties use it to filter old messages). If in W_1 the issuer of $\mathbf{tx}$ sends the same message that $\mathbf{tx}$ is generated at time τ , then in all honest parties' local views, W_3 is identical to the scenario in W_1 and they send the identical messages.

Finally, consider the case where both transaction issuers send messages claiming their transaction time. We show W_1 is indistinguishable from the world W_4 where $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ shall output $\mathbf{tx}' \prec \mathbf{tx}$. Consider the following scenario in W_4 : the transaction issuer of $\mathbf{tx}$ is corrupted and $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ assigns timestamp $\tau', \tau - \epsilon$ to transaction $\mathbf{tx}$ and $\mathbf{tx}'$ respectively. In addition, $\mathbf{tx}, \mathbf{tx}'$ are diffused to all parties except their senders at at the same time as those in W_1 ; the (honest) issuer of $\mathbf{tx}'$ sends messages saying that $\mathbf{tx}'$ is sent at time $\tau - \epsilon$ and the corrupted issuer of $\mathbf{tx}$ sends messages claiming $\mathbf{tx}$ is sent at time τ . Assume in W_1 , the issuer of $\mathbf{tx}'$ is corrupted; then, the honest issuer of $\mathbf{tx}$ sends messages claiming that $\mathbf{tx}$ is sent at time τ and the corrupted issuer of $\mathbf{tx}'$ sends messages claiming that $\mathbf{tx}'$ is sent at time $\tau - \epsilon$. In such a case, the diffusion pattern and messages sent in both W_1 and W_4 are identical. $\square$

Approximate order with any private functionality $\mathcal{F}$. Trusted hardware may be considered as a promising solution to transaction order fairness, as they are promised to help “authenticate and timestamp” a transaction that is generated even by a malicious host. Despite the fact that trusted execution environments are hard to synchronize, we show that when network delay exists, it leads to the imprecision on order fairness that cannot be circumvented by any powerful time-aware private functionalities.

We consider the model where there exists a functionality $\mathcal{F}_i$ for each party P_i which, can only communicate with P_i , global clock $\mathcal{G}_{\text{Clock}}$ and a correlated randomness functionality $\mathcal{G}_{\text{CR}}$; however $\mathcal{F}_i$ can function arbitrarily. Specifically, the correlated randomness setup $\mathcal{G}_{\text{CR}}$ returns a random string s from some distribution $\mathcal{D}$ to a functionality $\mathcal{F}$ only when $\mathcal{F}$ owns the code as specified in code in $\mathcal{G}_{\text{CR}}$; in such a way only the designated $\mathcal{F}$ can acquire the randomness. This can be

achieved, by considering the request message of format (cf. UC with global subroutine in [BCH⁺20]) $(\mu, (\text{sid}, \text{pid}))$ where μ specifies the sender's code.

Functionality $\mathcal{G}_{\text{CR}}$

This functionality is parameterized by a distribution $\mathcal{D}$.

- Upon receiving $\text{eid} = (\mu, (\text{sid}, \text{pid}))$ from P :
 1. If $\mu \neq \text{code}$ ignore this message.
 2. If there is no tuple of the form $(\text{sid}, \dots)$, generate $(s_1, \dots, s_n) \leftarrow \mathcal{D}(1^\lambda)$ and store $(\text{sid}, s_1, \dots, s_n)$. Otherwise, retrieve $(\text{sid}, s_1, \dots, s_n)$.
 3. Send $\text{Send}(\text{sid}, s_i)$ to P_i .

Functionality 2: *Correlated randomness*.

Given $\mathcal{G}_{\text{CR}}$, all responses parties acquired from $\mathcal{F}$ can be verified by other parties (in other words, they can be “convinced” by the messages from $\mathcal{F}$). We showcase that even if $\mathcal{F}$ can implement an arbitrary function and is time-aware (i.e., $\mathcal{F}$ learns the global time from $\mathcal{G}_{\text{Clock}}$), this does not help circumvent the impossibility result from the network delay. The subtlety in our proof, at a high level, is that corrupted parties can keep querying his local $\mathcal{F}$ for the same message in every round via different corrupted parties. We formalize this result in Theorem 2.

Theorem 2 (Impossibility of δ -approximate order with any private $\mathcal{F}$). *If $\Delta > 0$, then for any $\delta < \Delta$ there exist no protocol Π that securely realizes $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ parameterized with $\delta\text{-ApproxSenderOrder}$ in the $(\mathcal{G}_{\text{Clock}}, \mathcal{W}(\mathcal{G}_{\text{RO}}), \mathcal{F}_{\text{Diffuse}}, \mathcal{G}_{\text{CR}}, \mathcal{F})$ -hybrid environment.*

Proof. Suppose towards a contradiction, there exists a protocol Π that securely realizes $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ parameterized with $\delta\text{-ApproxSenderOrder}$ in the $(\mathcal{G}_{\text{Clock}}, \mathcal{W}(\mathcal{G}_{\text{RO}}), \mathcal{F}_{\text{Diffuse}}, \mathcal{G}_{\text{CR}}, \mathcal{F})$ -hybrid environment.

Consider two transactions tx, tx' in scenario W_1 . Let τ, τ' denote the timestamp that $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ assigns to them in the transaction buffer respectively. With out loss of generality, assume $\tau + \delta < \tau' < \tau + \Delta$ and tx' is generated by the adversary. Due to $\delta\text{-ApproxSenderOrder}$, $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ should output $\text{tx} \prec \text{tx}'$. We show this is indistinguishable from the scenario W_2 where $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ shall output $\text{tx}' \prec \text{tx}$. Before we go to W_2 , consider the responses from $\mathcal{F}$ in W_1 . The issuer of tx , when sending tx , can attach the message that he queries his own $\mathcal{F}$ at round τ ; the (corrupted) issuer of tx' , when sending tx' , however can attach the message that he queries his own $\mathcal{F}$ at round τ (but not τ'). This is because the adversary can keep querying tx' to $\mathcal{F}$ in every round up to τ' (which includes round $\tau < \tau'$). Since in each round, the adversary can query $\mathcal{F}$ via different corrupted parties (and these parties behave honestly for all other operations) the message that he acquired from $\mathcal{F}$ in round τ is admissible by all honest parties.

Now consider the following scenario W_2 : $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ assigns timestamp τ', τ to transaction tx and tx' respectively, and the transaction issuer of tx is corrupted. In addition, tx, tx' are diffused to all parties except their senders at at the same time in W_1 and W_2 (this diffusion pattern is possible, see the proof of Theorem 1). In W_2 , the (honest) issuer of tx' queries its private $\mathcal{F}$ the same time as it was sent messages hence get a response with respect to time τ ; meanwhile the corrupted issuer of tx keeps querying tx and diffuses tx with a response with respect to time τ at round τ' .

We learn that the diffusion pattern and the responses from $\mathcal{F}$ sent in both scenarios are identical, thus parties must decide the same order, which contradicts the fact that the protocol realizes $\delta\text{-ApproxSenderOrder}$. $\square$

5 Protocol Details

We describe our protocol $\Pi_{\text{Ledger}}^{\text{Fair}}$ in this section. The technical roadmap mainly consists two parts — transaction encryption and decryption using enclaves (in Section 5.1) and encrypted-transaction serialization (in Section 5.2). For the ease of presentation, we describe our protocol in the PoW and static setting (where the computational power is fixed yet known), we discuss how our protocol can be translated to different settings (PoS and dynamic participation) in Section 5.3. A full code specification of our protocol is presented in Section 5.4.

5.1 Transaction Encryption from Trusted Hardware

Our protocol follows the classical setting of state machine replication, where two types of parties — maintainers (miners/validators) and clients — are considered as protocol participants. We assume that all protocol maintainers are equipped with their own trusted hardware⁸. In contrary, a client, who sends transactions to the maintainers to use the ledger, is not equipped with an enclave.

The trusted hardware functionality $\mathcal{G}_{\text{att}}$. Following the model and treatment by Pass *et al.* [PST17], we capture the maintainers’ enclaves as a shared setup $\mathcal{G}_{\text{att}}$. This enclave functionality $\mathcal{G}_{\text{att}}$ is parameterized with a signature scheme SIG and a registered set of parties $\mathcal{P}$. Upon initialization, a master signing/verification key pair (msk, mvk) is generated, and parties can query the verification key via the GET-VK interface (i.e., it is assumed that in the real world such master verification key is securely distributed).

$\mathcal{G}_{\text{att}}$ has two “local” interfaces `INSTALL` and `RESUME`, and maintains a table T to record the installed program and their associated parties and internal states. Specifically, every registered party can install a program that can later be resumed by herself. Each program has its own memory which can only be reached and modified by the program itself. After resuming a program, the enclave signs the output and the executed program (with session id) using the master signing key msk ; later, any party can verify the output by using the master verification key queried from $\mathcal{G}_{\text{att}}$.

Functionality $\mathcal{G}_{\text{att}}$

The functionality is parameterized with a signature scheme $\text{SIG} = (\text{KeyGen}, \text{Sign}, \text{Verify})$ and a registered party set $\mathcal{P}$. On initialization, $(\text{msk}, \text{mvk}) \leftarrow \text{SIG.KeyGen}(1^\kappa)$ and $T \leftarrow \emptyset$.

- **Master key query.** Upon receiving `(GET-VK)` from P , send `(GET-VK, mvk)` to P .
- **Install an enclave.** Upon receiving `(INSTALL, idx, prog)` from some $P \in \mathcal{P}$:
 1. If P is honest, assert $idx = sid$.
 2. Generate nonce $eid \in \{0, 1\}^\kappa$, store $T[eid, P] = (idx, \text{prog}, \vec{0})$, send eid to P .
- **Resume an enclave.** Upon receiving `(RESUME, eid, inp)` from some $P \in \mathcal{P}$:
 1. Let $(idx, \text{prog}, \text{mem}) = T[eid, P]$, abort if not found.
 2. Let $(\text{outp}, \text{mem}) = \text{prog}(\text{inp}, \text{mem})$, update $T[eid, P] = (idx, \text{prog}, \text{mem})$.
 3. Let $\sigma = \text{SIG.Sign}(\text{msk}, idx, eid, \text{prog}, \text{outp})$, and send `(outp,  $\sigma$ )` to P .

Functionality 3: *The trusted execution environment.*

⁸Note that owning an enclave has no implication on the maintainer’s computational power. Our protocol works as long as the majority of the computational power is honest and a constant fraction of the enclaves are controlled by honest maintainers.

Transaction encryption scheme. We detail the code that $\mathcal{G}_{\text{att}}$ is expected to execute in TxDec (Program 1). This program is parameterized with a public key encryption scheme PKE and $\Lambda \in \mathbb{N}^+$ which indicates the time interval for hiding the transaction information and is counted by the number of blocks. TxDec has two interfaces, both can be executed for multiple times. The first interface KeyGen returns the public key pk in storage. When called for the first time, it runs the key generation algorithm of PKE and stores the key pair (pk, sk) . I.e., for each session, one enclave can only hold one public key pair. The second interface Dec, taking input a ciphertext ct and a chain $\mathcal{C}$, decrypts ct to a plain transaction tx when all the following conditions are satisfied: (i) ct successfully decrypts to $\text{tx} \parallel h$ where h is a valid block state in $\mathcal{C}$; and (ii) the length of $\mathcal{C}$, starting from the block with hash h has progressed for at least Λ blocks.

Importantly, in Dec there is a backdoor that can be triggered only when the decrypted transaction is an *all-zero* string. This backdoor, when triggered, re-decrypts the ciphertext using an alternate decryption algorithm Dec^* (which we detail very soon below). Looking ahead, this backdoor enables the simulator to equivocate fake transactions, as Dec^* allows for the adversary to program the random oracle and decrypt ct to an arbitrary string he chooses. Note that while every party can exploit this backdoor and equivocate her transaction, this is indeed harmless in the real world, because in order for a transaction to be considered legitimate, it should be proved by NIZKPoK as a valid transaction; though, an all-zero string can never be proved. In the ideal world, the simulator can generate a fake NIZK proof by hiding the information that random oracle is programmed from the black-box adversary, thus convincing him that an all-zero string is a valid transaction.

Program TxDec(PKE, Λ)

KeyGen <pre> 1: if (sk, pk) is not stored then 2: Generate $(\text{pk}, \text{sk}) \leftarrow \text{PKE.KeyGen}(1^\kappa)$ 3: end if 4: return pk.</pre>	Dec($ct, \mathcal{C}$) <pre> 5: Compute $\text{tx} \parallel h \leftarrow \text{PKE.Dec}(\text{sk}, ct)$ 6: if decryption fails then abort 7: if $\text{IsValidChain}^a(\mathcal{C}) = \text{false}$ then abort 8: if $\nexists \mathcal{B} \in \mathcal{C}$ s.t. $H(\mathcal{B}) = h$ then abort 9: Let ℓ denote the block height of $\mathcal{B}$ 10: if $\ell + \Lambda < \text{len}(\mathcal{C})$ then abort 11: if $\text{tx} = 0^{ \text{tx} }$ then $\text{tx} \parallel h \leftarrow \text{PKE.Dec}^*(\text{sk}, ct)$ 12: return (tx, ct).</pre>
--	---

^aDetails see Algorithm 5.

Program 1: *The enclave program for transaction decryption.*

Next we introduce the public-key encryption scheme PKE to be used in the TxDec program, consisting of four algorithms KeyGen, Enc, Dec and Dec^* . It is parameterized with a collection of one-way trapdoor functions and the restricted programmable and observable global random oracle $\mathcal{G}_{\text{rpoRO}}$ introduced by Camenish *et al.* [CDG⁺18]. Specifically, a trapdoor one-way function in $\mathcal{F}_{\text{owf}}$ consists of three functions f, f^{-1} and b which is the hardcore predicate of f .

- The key generation KeyGen algorithm randomly selects a trapdoor one-way function from $\mathcal{F}_{\text{owf}}$, and returns (f, b) as the public key and (f, f^{-1}, b) as the secret key.
- The encryption algorithm Enc takes a k -bit message as input. For the i -th bit, it samples a random element x_i in the input domain of f and computes ciphertext ct_i by concatenating $f(x_i)$ and $b(x_i) \oplus m_i$. After computing all ct_i it outputs the ciphertext $ct = ct_1, \dots, ct_k$.

- The decryption algorithm **Dec** to compute a k -bit plaintext works as follows. For the i -th bit and its corresponding ciphertext piece ct_i , m_i is computed by first extracting $x_i = f^{-1}(ct'_i)$ (ct'_i denotes all but the last bit), and then $b(x_i)$ is XORed with b'_i which is the last bit of ct_i to get m'_i . The plaintext is decrypted by concatenating all m'_i .
- The alternate decryption algorithm **Dec*** works exactly the same as **Dec**, except that it replaces the computation of hardcore bit $b(x_i)$ with the random oracle query, and the response is XORed with b'_i (where last bit is used as result).

Algorithm PKE($\mathcal{F}_{\text{owf}}, \mathcal{G}_{\text{rpoRO}}$)

Let $\mathcal{F}_{\text{owf}} = \{f_i : \mathcal{D}_i \rightarrow \mathcal{C}_i\}_{i \in I}$ be a collection of one-way trapdoor functions, where I and all $\mathcal{D}_i, \mathcal{C}_i$ have cardinality at least 2^κ . Let $f_i^{-1} : \mathcal{C}_i \rightarrow \mathcal{D}_i$ denote the trapdoor function with respect to f_i , and $b_i : \mathcal{D}_i \rightarrow \{0, 1\}$ denote the hardcore predicate of f_i .

KeyGen(1^κ)

- 1: Sample $i \xleftarrow{\$} I$.
- 2: Set $\text{pk} \leftarrow (f_i, b_i)$ and $\text{sk} \leftarrow (f_i, f_i^{-1}, b_i)$.
- 3: **return** (pk, sk) .

Enc(pk, m)

- 4: Parse pk as (f, b) and m as $m_1, \dots, m_k$.
- 5: **for** i **from** 1 **to** k **do**
- 6: Set $x_i \xleftarrow{\$} \mathcal{D}$
- 7: Set $ct_i \leftarrow f(x_i) \parallel (b(x_i) \oplus m_i)$
- 8: **end for**
- 9: Set $ct \leftarrow ct_1, \dots, ct_k$ and **return** ct .

Dec(ct)

- 10: Parse sk as (f, f^{-1}, b) and ct as $ct_1, \dots, ct_k$.
- 11: **for** i **from** 1 **to** k **do**
- 12: Parse ct_i as $ct'_i \parallel b'_i$
- 13: Compute $x'_i \leftarrow f^{-1}(ct'_i)$
- 14: Set $m'_i \leftarrow b(x'_i) \oplus b'_i$
- 15: **end for**
- 16: Set $m' \leftarrow m'_1, \dots, m'_k$ and **return** m' .

Dec*(ct)

- 17: Parse sk as (f, f^{-1}, b) and ct as $ct_1, \dots, ct_k$.
- 18: **for** i **from** 1 **to** k **do**
- 19: Parse ct_i as $ct'_i \parallel b'_i$
- 20: Compute $x'_i \leftarrow f^{-1}(ct'_i)$
- 21: Set $m'_i \leftarrow \mathcal{G}_{\text{rpoRO}}(x'_i) \oplus b'_i$
- 22: **end for**
- 23: Set $m' \leftarrow m'_1, \dots, m'_k$ and **return** m' .

Algorithm 4: Public-key encryption with two decryption modes.

We note that the first three algorithms (**KeyGen**, **Enc**, **Dec**) in **PKE** has actually instantiated a CPA-secure public-key encryption scheme. Nonetheless, the subtlety of using **PKE** in our protocol is that the secret key is stored in the enclave, and the simulator cannot control the parameter that the black-box adversary would like to query $\mathcal{G}_{\text{att}}$. Hence, equivocation in the ideal world should happen in a delicate fashion — by exploiting the last algorithm **Dec***, it provides a simple way for equivocating dummy ciphertexts without letting the enclave to directly interact with the simulator.

Remark 1. The **PKE** scheme can also be instantiated more efficiently using one-way trapdoor permutation. In particular, we can sample n hard-core bits from just one element x of the function domain by computing $b(x) \parallel b(f(x)) \parallel \dots \parallel b(f^{n-1}(x))$ [KO04].

Looking ahead, in the ideal world, the simulator can equivocate honest transaction by using the fact that only the senders (which are now dummy party) are allowed to equivocate their transactions and all other parties cannot distinguish whether a response (tx, ct) from $\mathcal{G}_{\text{att}}$ is decrypted by using **Dec** or **Dec***.

Lemma 3. No PPT distinguisher D , after observing polynomially many (m, ct) , can tell whether m is decrypted from ct by using **PKE.Dec** or **PKE.Dec*** with non-negligible probability.

Proof (Sketch). To prove the lemma we show that giving only the ciphertext and its decryption (we consider one bit message),

$$|\Pr_{b \leftarrow \{0,1\}}[\mathcal{A}(\text{Enc}(b), \text{Dec}(\text{Enc}(b))) = 1] \\ - \Pr_{b \leftarrow \{0,1\}}[\mathcal{A}(\text{Enc}(b), \text{Dec}^*(\text{Enc}(b))) = 1]| < \text{negl}(\kappa).$$

We first consider a new distinguisher D_1 trying to tell whether a plaintext is decrypted using PKE.Dec or $\text{PKE.Dec}'$ where in $\text{PKE.Dec}'$, the challenger returns decryption computed by randomly sample a bit and XOR it with the last bit of $\text{Enc}(b)$ (and he remembers this bit for later queries). We show that D_1 cannot distinguish with non-negligible probability, by showing a reduction to the security game of the hardcore bit. Roughly speaking, the adversary in the hardcore bit game runs D_1 ; if D_1 decides on PKE.Dec , then the adversary picks one (m, ct) outputs m XOR the last bit of c . If D_1 distinguishes with non-negligible probability, then the adversary finds $b(x)$ of $f(x)$ with probability non-negligibly more than one half, contradicting the security of hard-core bit.

We consider the next distinguisher D_2 trying to tell whether a plaintext is decrypted using PKE.Dec^* or $\text{PKE.Dec}'$. Again the advantage is negligible because when D_2 is given a ciphertext $ct = y \parallel b$, he cannot check whether the point $f^{-1}(y)$ is programmed in $\mathcal{G}_{\text{rpoRO}}$. Otherwise, a reduction to the security of one-way function can be made where the distinguisher of one-way function can invert and find pre-image with non-negligible probability. By combining these two we conclude the proof. $\square$

Enclave public-key selection. Since all maintainers are equipped with trusted hardware and each holds a public key, in the sense of communication complexity it is infeasible for a client to encrypt transactions to all the enclaves. To reduce the size of an encrypted transaction, the clients should pick a small subset of all public keys and only encrypt to these specific enclaves.

Arguably, as long as the client sends an encrypted transaction tx to at least one honest maintainer, it guarantees the correct and timely decryption of tx . Nonetheless, it is a non-trivial task to design a key subset selection scheme and the naïve solution to let clients pick public keys by themselves does not work. The reason is that when a transaction issuer takes full control over selecting public keys to encrypt to, the adversary can collude with her and select only the public keys of enclaves that are controlled by the adversary. Especially, the adversary can create different transactions and release them simultaneously to the network; later, he selectively opens one of them that profits him the most⁹.

To address this dilemma between the size of encrypted transactions and the security against selective decryption, we propose a new random subset selection scheme using $\mathcal{G}_{\text{att}}$. Specifically, protocol maintainers run a program `BlockTicket` parameterized with a pseudorandom function f_{PRF} . This program has only one interface which takes a κ -bit string h as input. Upon queried, it returns $f_{\text{PRF}}(\text{key}, h)$ where key is a stored PRF key (which is sampled uniformly upon initialization). For every maintainer, once they receive a valid block $\mathcal{B}$ with hash h , they generate a ticket (h, τ) and diffuse it to the network.

⁹This is also known as the selective opening problem, which is a major issue when using commitment to hide transaction contents. While this problem can be partially mitigated by employing a penalty scheme, under certain MEV circumstances the revenue of selective opening can be much higher than the penalty.

Program BlockTicket(f_{PRF})

TicketGen(h)

- 1: **if** key is not stored **then** generate $\text{key} \xleftarrow{\$} \{0, 1\}^\kappa$ and store key.
- 2: Set $\tau \leftarrow f_{\text{PRF}}(\text{key}, h)$ and **return** (h, τ) .

Program 2: *The enclave program for ticket generation.*

We then consider a function $f_{\text{select}}(\mathcal{S}, m, \tau)$ which, taking a set $\mathcal{S}$, a threshold $m \in \mathbb{N}^+$ and $\tau \in \{0, 1\}^\kappa$ as parameters, outputs an element in $\mathcal{S}$. Formally, let $\binom{\mathcal{S}}{m}$ denote the (ordered) set of all m -combinations of $\mathcal{S}$ (when $|\mathcal{S}| < m$, f_{select} returns $\mathcal{S}$ directly), we define the selection function as

$$f_{\text{select}} = \binom{\mathcal{S}}{m} \left[\tau \bmod \left| \binom{\mathcal{S}}{m} \right| \right].$$

Looking ahead, when a client is to issue a transaction tx , she picks a block hash h on (the settled part of) her local chain $\mathcal{C}_{\text{loc}}$ and encrypts tx concatenated with h . Then, she selects a ticket (h, τ) (from the set of all tickets $(h, \cdot)$ that she has) and applies f_{select} on the set of public keys published on the blockchain up to block with hash h , $m = \text{polylog} \kappa$ which will be a parameter of the protocol and the ticket (h, τ) .

We provide some intuition on why this block ticket scheme underpins a secure subset selection over the public keys. Notice that $m = \text{polylog} \kappa$ and the public keys of enclaves controlled by honest parties account for a constant fraction of all the public keys. By randomly selecting an element in $\binom{\mathcal{S}}{m}$, the probability that the chosen m -combination contains no honest public key is negligible with respect to the security parameter κ . Furthermore, since only a ticket that is associated with a settled block will be used by parties to select public keys (otherwise the encrypted transaction will be considered illegitimate), the total number of valid tickets throughout an execution running for $L = \text{poly}(\kappa)$ rounds is polynomially bounded by κ and n where n is the number of enclaves. Refer to Lemma 7 for the formal analysis of this scheme.

Remark 2. Our public-key subset selection scheme still allows transaction issuers to select the ticket that she prefers. I.e., when the issuer chooses not to trust some enclaves, she can select tickets that does not contain their public keys. This can be useful in the model where some enclaves can be compromised, but later honest parties get notified and avoid using their public keys. In such a way the system self-heals from temporarily losing input causality.

Issuing a transaction. We elaborate on how a client can issue a transaction, the code description of this procedure is presented in Protocol 4 in Section 5.4. Recall that clients are not equipped with trusted hardware thus they should encrypt their transactions using the maintainers' public keys. They can acquire the master verification key mvk from $\mathcal{G}_{\text{att}}$, and the maintainers will publish their public key on the blockchain. Further, by listening to the network they get tickets for every block in the settled part of the blockchain¹⁰.

In order to send a transaction tx , the client first picks a block $\mathcal{B}$ in the settled part of her local chain; i.e., $\mathcal{B} \leftarrow \text{head}(\mathcal{C}_{\text{loc}}^{\lceil k})$. Let h denote the hash of $\mathcal{B}$ and $\mathcal{S}_h$ the set of tickets associated with h .

¹⁰Note that our protocol requires an initial enclave public key registration stage to gather sufficiently many on-chain honest enclave public keys. This requires the clients to wait until the blockchain grows to at least $2k$ blocks (where k is the common prefix parameter) so that at least one honest block gathers honest public keys and gets buried sufficiently deep in the settled blockchain. This “warm-up” stage can be captured by enforcing the transaction process mechanism to start after the chain grows to $2k$ blocks.

The client first picks a $(h, \tau) \in \mathcal{S}_h$ randomly, and then selects the public keys $\mathcal{S}_{\text{pk}}^\tau \leftarrow f_{\text{select}}(\mathcal{S}_{\text{pk}}, m, \tau)$ where $\mathcal{S}_{\text{pk}}$ denotes the set of on-chain enclave public keys up to block $\mathcal{B}$. Let $m' = |\mathcal{S}_{\text{pk}}^\tau|$. The client then encrypts $\mathbf{tx}$ as $ct = (ct_1, ct_2, \dots, ct_{m'})$ such that

$$ct_i = \text{PKE.Enc}(\text{pk}_i, (\mathbf{tx} \parallel h)) \text{ where } \text{pk}_i = \mathcal{S}_{\text{pk}}^\tau[i]$$

using the encryption algorithm in Algorithm 4. Note that $\mathbf{tx}$ denotes both the transaction content and metadata and they are all encrypted¹¹, hence each ct_i is indistinguishable from a random string and can only be decrypted using the trusted hardware with public key pk_i .

In order to prevent the adversary from issuing “bad” transactions (e.g., different ct_i encrypts different transactions, or $\mathbf{tx}$ is not a valid transaction), a NIZKPoK proof should be attached. Our statement is of form $\text{stmt} = (h, \mathbf{st}, (ct_1, \dots, ct_m), (\text{pk}_1, \dots, \text{pk}_m))$ and witness $\mathbf{w} = (\mathbf{tx}, (r_1, \dots, r_m))$. The NP relation is defined as follows (where we slightly abuse the notation and let Enc taking an additional parameter r to replace the randomness generation by using r):

$$\forall i \in [m], \exists (r_i, \mathbf{tx}) \text{ s.t. } ct_i = \text{PKE.Enc}(\text{pk}_i, (\mathbf{tx} \parallel h), r_i) \text{ and } \mathcal{Q}_{\text{validTX}}(\mathbf{tx}, \mathbf{st}) = 1. \quad (2)$$

Note that to simplify the transaction validation, we write $\mathcal{Q}_{\text{validTX}}$, taking input a transaction $\mathbf{tx}$ and a blockchain state $\mathbf{st}$, as the predicate such that $\mathcal{Q}_{\text{validTX}}(\mathbf{tx}, \mathbf{st}) = 1$ if and only if $\mathbf{tx}$ is a valid transaction with respect to state $\mathbf{st}$.

We elaborate on the NIZKPoK scheme that we are going to use. Let Π_{NIZK} denote a NIZKPoK protocol [LR22] consisting of six algorithms **Setup**, **Prove**, **Verify**, **SimSetup**, **SimProve** and **Extract**. Especially, Π_{NIZK} satisfies three properties, namely (i) overwhelming completeness; (ii) non-interactive multiple special honest-verifier zero-knowledge; and (iii) non-interactive special simulation-soundness. Note that Π_{NIZK} can be implemented in the $\mathcal{G}_{\text{rpoRO}}$ model.

After selecting the block ticket τ , the set of public keys $\mathcal{S}_{\text{pk}}^\tau$ and generating the ciphertexts ct , the client then generates a NIZK proof $\pi = \Pi_{\text{NIZK}}.\text{Prove}((h, \mathbf{st}, ct, \mathcal{S}_{\text{pk}}^\tau), (\mathbf{tx}, r))$ where $\mathbf{st}$ is the (hash of) blockchain state with respect to h . The client now is ready to issue the new transaction by diffusing (π, ct, τ) to the network.

Transaction decryption. To open a transaction associated with block hash h , a chain mined from h , progressing for at least Λ blocks, should be provided. We note that while in our description, clients should select a block in the immutable part of the blockchain (thus h will be extended with overwhelming probability), there is actually no restrictions on the clients to prevent them from selecting a very recent block. However, this comes with the risk of hurting liveness — if a block $\mathcal{B}$ in the unsettled part is selected for issuing a transaction and $\mathcal{B}$ get orphaned, then that transaction cannot be considered legitimate on this chain.

While a transaction associated with an orphaned block won’t be accepted on the main chain, we highlight that all valid transactions can be opened eventually, when there is a curious adversary that works on the orphaned fork alone and hence open the encryption using his own enclave.

5.2 Blind Transaction Serialization

In this section we present our PoW blockchain framework and show how a virtual ledger $\mathcal{L}$ can be built on top of the chain $\mathcal{C}$. In a nutshell, the blockchain $\mathcal{C}$ can be seen as derived from the Nakamoto-style PoW chain by additionally binding the mining procedure of the blocks that forms the chain $\mathcal{C}$ and a new type of block called “profile blocks”, using 2×1 (pronounced “two-for-one”)

¹¹Previous works encrypt only transaction content but metadata (e.g., transaction issuer’s public key) remains transparent.

PoW. Note that the 2×1 PoW blockchain has been proposed in [GKL15]. We borrow their basic constructions but adapt it specifically to reach agreement over the timestamp of each transaction. The virtual ledger $\mathcal{L}$ is built by serializing all transactions based on their aggregated timestamp.

In 2×1 PoW, a hash function output u is checked twice for u and its reversed bit-string $[u]^R$. If $u < T$, a block that extends the chain $\mathcal{C}$ is produced; and if $[u]^R < T$, a new profile block PB is mined (which we will detail soon). In such a way, the mining procedure of the chain $\mathcal{C}$ and profile blocks are “bound” together and the adversary cannot gain advantage on either type of blocks by dropping from mining the other.

The blockchain $\mathcal{C}$ in $\Pi_{\text{Ledger}}^{\text{Fair}}$ is a sequence of blocks connected by the hash reference to the previous block, and all validation and chain-selection algorithms follow the longest-chain rule. The only difference between our chain $\mathcal{C}$ in $\Pi_{\text{Ledger}}^{\text{Fair}}$ and that in, e.g., Bitcoin, is that blocks in $\mathcal{C}$ does not include transactions “directly”. Instead, they only include valid profile blocks and valid enclave public keys (also, valid transaction decryption signed by the enclave). As for the profile blocks, they share the same block head structure with on-chain blocks; further, for each PB, its content is a transaction list of pair (tag, t) where tag is the transaction identifier (hash of the tuple of encrypted transaction, NIZK proof and block ticket (ct, π, τ)) and t is its local receiving time.

When a maintainer P receives a transaction in the form of (ct, π, τ) , it first extracts the block hash h associated with this transaction. If h is not on P ’s local chain, the transaction gets rejected. Further, if the proof π fails to verify, or it contains a state st does not match h , or ct is encrypted from the wrong set of enclave public keys associated with block ticket τ , P rejects this transaction either. When all these verification pass, P bookkeeps the current time r with this transaction in its buffer as $((ct, \pi, \tau), r)$

Transaction timestamping. To facilitate transaction timestamping via profile blocks, we employ two additional parameters for validating profile blocks — the recency parameter R which is used to guarantee the freshness of profile blocks and the profile window length parameter K . Specifically, a profile block PB should “attach” to a valid block $\mathcal{B}$ in the settled part of the blockchain (by including $\mathcal{B}$ ’s block hash in its header), as the freshness proof that PB is not mined too early ago. Meanwhile, the blockchain $\mathcal{C}$ should reject stale profile blocks (to prevent the adversary from withholding them and releasing in the very future). Let ℓ denote the height of block $\mathcal{B}$ that PB attaches to. PB is considered as a valid profile block only when PB is included in a block with height less than $\ell + R$.

We detail the transaction settlement and timestamping scheme. Consider a transaction tx (at this stage miners only know its unique identifier tag) and $\mathcal{B}$ the first block that contains a profile block PB with entry $(\text{tag}, \cdot)$. Let ℓ denote the block height of $\mathcal{B}$. The timestamp of tx (or, its position in the final ledger) is computed by calculating the median timestamps of tag from all profile blocks included in blocks with height at most $\ell + K$. Formally,

$$\text{TS}(\text{tag}) \triangleq \text{med}\{t \mid (\text{tag}, t) \in \text{PB} \in B \wedge \ell \leq \text{height}(B) < \ell + K\}.$$

Note that if a profile block in the K -block window does not report an entry $(\text{tag}, \cdot)$, it is counted as reporting $(\text{tag}, +\infty)$ (i.e., the miner had never received tag thus cannot decrypt it). In other words, in order for a transaction tx with identifier tag to be included in the ledger $\mathcal{L}$, the majority of these profile blocks should report an entry $(\text{tag}, \cdot)$, otherwise the computed timestamp is ∞ meaning tag fails to settle into the ledger¹².

Security guarantees. Looking ahead, in our analysis we show that our blockchain $\mathcal{C}$ provides all security guarantees as the conventional ones, with additional good properties on the profile

¹²Under this majority rule, even if the adversary with minority of computational power completely drops from mining a transaction tag , honest parties can still settle tag on their own with a good timestamp.

blocks. In particular, for any transaction $\mathbf{tx}$ and its associated K -block window, at least half of the profile blocks in this window are produced by honest parties. Given the overwhelming probability on winning the majority of profile blocks, the ledger built on top of $\mathcal{C}$ is a consistent and live one with two additional properties — sender fairness and input causality. In terms of sender fairness, for each transaction $\mathbf{tx}$ that enters the network at round r , eventually our ledger will finalize a timestamp t such that $r \leq t \leq r + \Delta$ for $\mathbf{tx}$. Further, regarding input causality, since no party except the transaction issuer can learn (any information about) $\mathbf{tx}$ before the chain extends for at least $\Lambda > K$ blocks, the adversary can only issue transactions after $\mathbf{tx}$ has got finalized.

5.3 Further Discussions

In this section we discuss how our protocol can be naturally translated to a Nakamoto-style Proof-of-Stake protocol, and how it can operate with dynamic participation.

Proof-of-Stake instantiation. Our protocol turns out to have a natural adaption to the PoS setting with a few minor adaptations. Note that the transaction encryption mechanism that we employ are blockchain-agnostic hence no additional modification is required. We hence focus on the blockchain part.

On one hand, we shall instantiate the 2×1 PoW blockchain using PoS. This translation is immediate: the 2×1 PoW can be replaced by using two independent VRF evaluations, one for extending the PoS chain and the other for generating profile blocks. This replacement is provably equivalent and has seen wide usage (see, e.g., [FGKR20] for more discussion). On the other hand, in PoS the simulation for chain extending is “for free”, and since the enclaves are clock-oblivious they should not decrypt transactions using the “progression” on a PoS chain. To address this problem, we point to the “density”-based chain selection rule in Ouroboros Genesis [BGK⁺18], where the selection of two chains with long forks is based on selecting the denser chain after the fork, capturing the fact that honest parties win more slots than the adversary. A similar approach can be applied in the enclave decryption program: in order to decrypt a transaction $\mathbf{tx}$, a chain extending from the state bound with $\mathbf{tx}$ should be provided, with sufficiently many blocks in each interval (i.e., the chain should be dense), until some pre-defined time length.

Dynamic participation. Our protocol can operate with dynamic participation, with certain modifications that we provide a high-level intuition. The 2×1 PoW blockchain that we use can go with fluctuating computational power (the 2×1 mining with difficulty adjustment has been used and proved secure in, e.g., [KLS24]). Further, the enclave program can be modified to decrypt transactions if certain amount of block difficulty has been accumulated since the state that transaction bound to. The main challenge is when maintainers come and go, the set of enclave public keys that clients are supposed to use should be updated periodically. Notice that in [KLS24] the mining difficulty is adjusted by epochs, hence we could employ an additional public key registration phase for each epoch, and based on the block that a transaction is bound with, it selects the appropriate set of keys to apply the ticket selection — and of course, this requires a more refined parametrization.

5.4 A Full Protocol Description

We introduce the main $\Pi_{\text{Ledger}}^{\text{Fair}}$ protocol instance that dispatches to the relevant subprocesses. This protocol is parameterized with the k the common prefix, R the recency parameter, m the public-key subset size and K the profile window length. Refer to Table 1 in Section B for a detailed explanation.

Protocol $\Pi_{\text{Ledger}}^{\text{Fair}}$

Global Variables:

- Parameters: k, R, m, K
- Local states: $r, \mathcal{C}_{\text{loc}}, \text{buffer}_{\text{tx}}, \text{buffer}_{\text{PB}}, \text{buffer}_{\text{pk}}$

Registration/Deregistration:

- Upon receiving $(\text{REGISTER}, \mathcal{R})$, where $\mathcal{R} \in \{\mathcal{G}_{\text{Ledger}}^{\text{Fair}}, \mathcal{G}_{\text{Clock}}, \mathcal{G}_{\text{RO}}, \mathcal{G}_{\text{rpoRO}}, \mathcal{G}_{\text{att}}\}$, execute $\text{Registration}(\text{P}, \text{sid}, \text{Reg}, \mathcal{R})$.
- Upon receiving $(\text{DE-REGISTER}, \mathcal{R})$, where $\mathcal{R} \in \{\mathcal{G}_{\text{Ledger}}^{\text{Fair}}, \mathcal{G}_{\text{Clock}}, \mathcal{G}_{\text{RO}}, \mathcal{G}_{\text{rpoRO}}, \mathcal{G}_{\text{att}}\}$, execute $\text{Deregistration}(\text{P}, \text{sid}, \text{Reg}, \mathcal{R})$.
- Upon receiving input $(\text{IS-REGISTERED}, \text{sid})$ return $(\text{REGISTER}, \text{sid}, 1)$ if the local registry Reg indicates that this party has successfully completed a registration with $\mathcal{R} = \mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ (and did not de-register since then). Otherwise, return $(\text{REGISTER}, \text{sid}, 0)$.

Interacting with the Ledger: Upon receiving a ledger-specific input $I \in \{(\text{SUBMIT}, \dots), (\text{READ}, \dots), (\text{MAINTAIN-LEDGER}, \dots)\}$ verify first that all resources are available. If not all resources are available, then ignore the input; else (i.e., the party is operational and time-aware) execute one of the following steps depending on the input I :

- If $I = (\text{SUBMIT}, \text{sid}, \text{tx})$ then invoke $\text{IssueNewTransaction}(\text{P}, \text{sid}, \text{tx})$.
- If $I = (\text{MAINTAIN-LEDGER}, \text{sid}, \text{minerID})$ then invoke $\text{LedgerMaintenance}(\mathcal{C}_{\text{loc}}, \text{P})$; if LedgerMaintenance halts then halt the protocol execution (all future input is ignored).
- If $I = (\text{READ}, \text{sid})$ then invoke $\text{ReadState}(\text{P}, \text{sid})$.

Handling external calls:

- Upon receiving $(\text{CLOCK-READ}, \text{sid}_C)$ forward it to $\mathcal{G}_{\text{Clock}}$ and output $\mathcal{G}_{\text{Clock}}$'s response.
- Upon receiving $(\text{CLOCK-UPDATE}, \text{sid}_C)$, record that a clock-update was received in the current round. If this protocol instance is currently only registered to the clock (and no other functionality), then forward $(\text{CLOCK-UPDATE}, \text{sid}_C)$ to $\mathcal{G}_{\text{Clock}}$.

Protocol 1: *The main protocol instance of $\Pi_{\text{Ledger}}^{\text{Fair}}$.*

Registration and de-registration. In order to participate in the protocol, parties need to register with their resources. Protocol 2 captures the registration procedure. Note that before registering with the network and random oracle, parties should check if they have access to all global functionalities.

Protocol $\text{Registration}(\text{P}, \text{sid}, \text{Reg}, \mathcal{G})$

- 1: **if** $\mathcal{G} \in \{\mathcal{G}_{\text{Clock}}, \mathcal{G}_{\text{att}}\}$ **then**
- 2: Send $(\text{REGISTER}, \text{sid})$ to $\mathcal{G}$, set registration status to registered with $\mathcal{G}$, and output the valued received by $\mathcal{G}$.
- 3: **else if** $\mathcal{G} = \mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ **then**
- 4: **if** the party is not registered with $\mathcal{G}_{\text{Clock}}, \mathcal{G}_{\text{RO}}, \mathcal{G}_{\text{rpoRO}}$ or $\mathcal{G}_{\text{att}}$ or is already registered with all setup functionalities **then**
- 5: ignore this input
- 6: **else**
- 7: Send $(\text{REGISTER}, \text{sid})$ to $\mathcal{F}_{\text{Diffuse}}$.
- 8: Output $(\text{REGISTER}, \text{sid}, \text{P})$ once completing the registration with all the above resources.
- 9: **end if**

10: **end if**

Protocol 2: *The registration procedure of $\Pi_{\text{Ledger}}^{\text{Fair}}$.*

The de-registration procedure (Protocol 3) is analogous to the above, where de-registering from the ledger is simplified as dropping from the network functionality.

Protocol Deregistration(P, sid, Reg, $\mathcal{G}$)

- 1: **if** $\mathcal{G} \in \{\mathcal{G}_{\text{Clock}}, \mathcal{G}_{\text{RO}}, \mathcal{G}_{\text{rpoRO}}, \mathcal{G}_{\text{att}}\}$ **then**
- 2: Send (DE-REGISTER, sid) to $\mathcal{G}$, set registration status as de-registered with $\mathcal{G}$, and output the valued received by $\mathcal{G}$.
- 3: **end if**
- 4: **if** $\mathcal{G} = \mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ **then**
- 5: Send (DE-REGISTER, sid) to $\mathcal{F}_{\text{Diffuse}}$, set its registration status as de-registered with $\mathcal{F}_{\text{Diffuse}}$ and output (DE-REGISTER, sid, P).
- 6: **end if**

Protocol 3: *The registration procedure of $\Pi_{\text{Ledger}}^{\text{Fair}}$.*

The transactions issuing procedure. In order to issue a transaction $\mathbf{tx}$, party P needs to encrypt $\mathbf{tx}$ using a subset of on-chain public keys, computed from a ticket associated with a block in the settled blockchain. Additionally, a NIZKPoK that proves the validity of $\mathbf{tx}$ and the correctness of each ciphertext should be attached (See the relation in Equation (2)).

Protocol IssueNewTransaction(P, sid, $\mathbf{tx}$)

- 1: $h \leftarrow \text{head}(\mathcal{C}_{\text{loc}}^k)$ and $\mathbf{st} \leftarrow$ blockchain state associated with h
- 2: $\mathcal{S}_{\text{pk}} \leftarrow$ all enclave public keys up to block with hash h
- 3: $\mathcal{S}_{\text{pk}}^\tau \leftarrow f_{\text{select}}(\mathcal{S}_{\text{pk}}, m, \tau)$ and $m' \leftarrow |\mathcal{S}_{\text{pk}}^\tau|$
- 4: **for** i **from** 1 **to** m' **do**
- 5: $\text{pk}_i \leftarrow \mathcal{S}_{\text{pk}}^\tau[i]$, $r_i \xleftarrow{\$} \{0, 1\}^\kappa$
- 6: $ct_i \leftarrow \text{PKE.Enc}(\text{pk}_i, \mathbf{tx}, h; r_i)$
- 7: **end for**
- $\triangleright$ Generate NIZK proof.
- 8: $\pi \leftarrow \Pi_{\text{NIZK}}.\text{Prove}((h, \mathbf{st}, (ct_1, \dots, ct_{m'}), (\text{pk}_1, \dots, \text{pk}_{m'})), (\mathbf{tx}, (r_1, \dots, r_{m'})))$
- 9: $\text{buffer}_{\text{tx}} \leftarrow \text{buffer}_{\text{tx}} \parallel (\pi, ct_1, \dots, ct_{m'})$
- 10: Send (diffuse, sid, $(\pi, ct_1, \dots, ct_{m'}, \tau)$) to $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$

Protocol 4: *The procedure to issue a new transaction in $\Pi_{\text{Ledger}}^{\text{Fair}}$.*

Fetch information. Parties fetch information from different diffusion functionalities — $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$, $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$, $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$, $\mathcal{F}_{\text{Diffuse}}^{\text{pk}}$ — to learn the new chains, profile blocks, transactions and enclave public keys, respectively. When receiving an encrypted transaction, they verify if it provides a valid block

ticket and whether it attaches a good NIZK proof. If the transaction verifies, book-keep its local receiving time.

Protocol FetchInformation(P, sid)

- ▷ Fetch blocks.
- 1: Send (FETCH, sid) to $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$; denote the response from $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ by (FETCH, sid, b).
- 2: Extract chains $\mathcal{C}_1, \dots, \mathcal{C}_k$ from b.
- 3: $\mathcal{C}_{\text{loc}} \leftarrow \text{maxvalid}(\mathcal{C}_{\text{loc}}, \mathcal{C}_1, \dots, \mathcal{C}_k)$
- ▷ Fetch profile blocks.
- 4: Send (FETCH, sid) to $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$; denote the response from $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$ by (FETCH, sid, b).
- 5: Extract profile blocks $\text{PB}_1, \dots, \text{PB}_k$ from b.
- 6: Set $\text{buffer}_{\text{PB}} \leftarrow \text{buffer}_{\text{PB}} \parallel (\text{PB}_1, \dots, \text{PB}_k)$
- ▷ Fetch enclave public keys and tickets.
- 7: Send (FETCH, sid) to $\mathcal{F}_{\text{Diffuse}}^{\text{pk}}$; denote the response from $\mathcal{F}_{\text{Diffuse}}^{\text{pk}}$ by (FETCH, sid, b).
- 8: Extract all public keys $\text{pk}_1, \dots, \text{pk}_k$ from b.
- 9: Set $\text{buffer}_{\text{pk}} \leftarrow \text{buffer}_{\text{pk}} \parallel (\text{pk}_1, \dots, \text{pk}_k)$
- ▷ Fetch encrypted transactions.
- 10: Send (FETCH, sid) to $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$; denote the response from $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$ by (FETCH, sid, b).
- 11: Extract transactions $(\pi_1, \text{ct}_1, \tau_1), \dots, (\pi_k, \text{ct}_k, \tau_k)$ from b.
- 12: **for** i **from** 1 **to** k **do**
- 13: **if** τ_i is valid **and** $\Pi_{\text{NIZK}}.\text{Verify}(\pi) = 1$ **and** $h, st \in \pi$ match that on $\mathcal{C}_{\text{loc}}$ **then**
- 14: $\text{buffer}_{\text{tx}} \leftarrow \text{buffer}_{\text{tx}} \parallel ((\pi_i, \text{ct}_i, \tau_i), \mathbf{r})$ ◁ Bookkeep local receiving time $\mathbf{r}$
- 15: **end if**
- 16: **end for**

Protocol 5: *The fetch-information procedure in $\Pi_{\text{Ledger}}^{\text{Fair}}$.*

Chain validation rules. The validation procedure (Algorithm 5) generally follows that in Bitcoin backbone protocol, plus additionally verifying the validity and freshness of profile blocks.

For simplicity, we use validBlock^T to verify if a block is a successful PoW.

$$\text{validBlock}^T(\text{ctr}, r, h, st, h', st') \stackrel{\text{def}}{=} H(\text{ctr}, r, h, st, h', \text{val}) < T \wedge \text{ctr} < 2^{32}.$$

Similarly, we use validProfile^T to verify if a profile block is a successful PoW by checking the reverse of the string.

Algorithm IsValidChain($\mathcal{C}$)

- 1: **if** $\mathcal{C}$ starts with a block other than **G** **then return false**
- 2: **if** $\mathcal{C}$ encodes an invalid state with $\text{invalidstate}(\vec{st}) = 0$ **then return false**
- 3: **if** $\mathcal{C}$ contains profile blocks with invalid content **then return false**
- 4: $r' \leftarrow \mathbf{r}, \mathcal{B} \leftarrow \text{head}(\mathcal{C}), h^* \leftarrow H(\mathcal{B})$
- 5: **while** $\mathcal{C} \neq \varepsilon$ **do**
- 6: $\text{isValid} \leftarrow \text{true}$
- 6: ▷ Check validity of block header
- 7: **if** $(\text{validBlock}^T(\mathcal{B}) = \text{false}) \vee (h^* \neq H(\mathcal{B})) \vee (\text{TS}(\mathcal{B}) \geq r')$ **then** $\text{isValid} \leftarrow \text{false}$
- 7: ▷ Check validity of profile blocks in $\mathcal{B}$
- 8: **for** $\text{PB} \in \mathcal{B}$ **do**
- 9: Parse PB as $\langle \cdot, r, \cdot, \cdot, h', \cdot \rangle$

```

10:    $\mathcal{B}' \leftarrow$  the block in  $\mathcal{C}$  s.t.  $H(\mathcal{B}) = h'$ 
    ▷ Check validity of profile header
11:   if ( $\text{validProfile}^T(\text{PB}) = \text{false}$ )  $\vee r \geq \text{TS}(\mathcal{B})$  then  $\text{isValid} \leftarrow \text{false}$ 
12:   if ( $\mathcal{B}' = \varepsilon$ )  $\vee r \geq \text{TS}(\mathcal{B}') + R$  then  $\text{isValid} \leftarrow \text{false}$            ◁ Check freshness and recency
13:   end for
14:   if  $\text{isValid} = \text{true}$  then
15:      $r' \leftarrow r, h^* \leftarrow h$ 
16:     Remove the rightmost block in  $\mathcal{C}$ 
17:      $\mathcal{B} \leftarrow \text{head}(\mathcal{C})$ 
18:   else
19:     return false
20:   end if
21: end while
22: return true

```

Algorithm 5: The chain validation rule in $\Pi_{\text{Ledger}}^{\text{Fair}}$.

Longest chain selection. Parties use the same longest-chain rule (Algorithm 6) as the Bitcoin backbone protocol to select their working chain.

Algorithm maxvalid($\mathcal{C}_1, \dots, \mathcal{C}_k$)

```

1:  $\mathcal{C}_{\max} \leftarrow \varepsilon$ 
2: for  $i$  from 1 to  $k$  do
3:   if  $\text{IsValidChain}(\mathcal{C}_i)$  and  $\text{len}(\mathcal{C}_i) > \text{len}(\mathcal{C}_{\max})$  then  $\mathcal{C}_{\max} \leftarrow \mathcal{C}_i$ 
4: end for
5: return  $\mathcal{C}_{\max}$ 

```

Algorithm 6: The chain selection rule in $\Pi_{\text{Ledger}}^{\text{Fair}}$.

The mining procedure. Parties use 2×1 PoW to extend the blockchain and mine new profile blocks. If they succeed on either procedure, they diffuse the extended chain and new profile blocks to the corresponding diffusion network.

Protocol MiningProceudre(P, sid)

```

▷ The following steps are executed in an (MAINTAIN-LEDGER, sid, minerID)-interruptible manner:
1: Set  $st \leftarrow$  Merkle root of profile blocks in  $\text{buffer}_{\text{PB}}$ , decrypted transactions in  $\text{buffer}_{\text{tx}}$  and public keys in  $\text{buffer}_{\text{pk}}$  that are not mined in  $\mathcal{C}_{\text{loc}}$ 
2: Set  $st' \leftarrow$  Merkle root of  $((\pi_1, t_1, \tau_1), \dots, (\pi_k, t_k, \tau_k))$  where  $\pi_i$  is a transaction that is not settled in  $\mathcal{C}_{\text{loc}}$  and  $t_i$  its local receiving time
3:  $h \leftarrow H(\text{head}(\mathcal{C}_{\text{loc}})), h' \leftarrow H(\text{head}(\mathcal{C}_{\text{loc}}^k))$ 
4:  $u \leftarrow H(\text{ctr}, \mathbf{r}, h, h', st, st')$ 
5: if  $u < T$  then
6:    $\mathcal{B} \leftarrow \langle \text{ctr}, h, h', \mathbf{r}, st, st' \rangle$  and  $\mathcal{C}_{\text{loc}} \leftarrow \mathcal{C}_{\text{loc}} \parallel \mathcal{B}$ 

```

```

7:   Send (DIFFUSE, sid,  $\mathcal{C}_{\text{loc}}$ ) to  $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$  and proceed from here upon next activation of this procedure
8: end if
9: if  $[u]^R < T$  then
10:    $\text{PB} \leftarrow \langle \text{ctr}, h, h', \mathbf{r}, st, st' \rangle$ 
11:   Send (DIFFUSE, sid, PB) to  $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$  and proceed from here upon next activation of this procedure
12: end if
13:  $\text{ctr} \leftarrow \text{ctr} + 1$ 

```

Protocol 6: *The mining procedure in $\Pi_{\text{Ledger}}^{\text{Fair}}$.*

Ledger Maintenance. We group all the steps in the main ledger operation in LedgerMaintenance.

Protocol LedgerMaintenance(P, sid)

▷ The following steps are executed in an (MAINTAIN-LEDGER, sid, minerID)-interruptible manner:

```

1: Invoke FetchInformation(sid, P) to receive the newest messages for this round
   ▷ Decrypt transactions
2: for  $(\pi, ct, \tau) \in \mathcal{C}_{\text{loc}}$  do
3:   if  $\text{len}(\mathcal{C}_{\text{loc}}) > \mathcal{B} + \Lambda$  where  $\mathcal{B}$  is a block with hash  $h$  in  $\pi$  and  $\text{pk}_i \in \pi$  is miner's enclave public key then
4:     Send (TX-DEC,  $ct_i, \mathcal{C}_{\text{loc}}$ ) to  $\mathcal{G}_{\text{att}}$  and get response  $(\mathbf{tx}, ct, \sigma)$ 
5:     Add  $(\mathbf{tx}, ct, \sigma)$  to  $\text{buffer}_{\mathbf{tx}}$ 
6:   end if
7: end for
8: if  $t_{\text{work}} < \tau$  then
9:   Call MiningProcedure(P, sid)
10:  Set  $t_{\text{work}} \leftarrow \tau$ 
11: end if
12: while A (CLOCK-UPDATE, sidC) has not been received during the current round do
13:   Give up activation (set the anchor here)
14: end while
15: Send (CLOCK-UPDATE, sidC) to  $\mathcal{G}_{\text{Clock}}$ .

```

◁ Party will lose its activation here

Protocol 7: *The ledger maintenance procedure in $\Pi_{\text{Ledger}}^{\text{Fair}}$.*

Reading the state. Upon receiving a READ command, parties extract a transaction list from their local chain $\mathcal{C}_{\text{loc}}$. Recall that transaction inclusion and consensus are decoupled, we first introduce an algorithm ExtractTransactionSequence such that, taking input a chain $\mathcal{C}$, converts it to a sequence of blocks of transactions $\vec{N}$, which are extracted using the median timestamp. Note that $\vec{N}$ shares the same length as $\mathcal{C}$.

Algorithm ExtractTransactionSequence($\mathcal{C}$)

```

1: Initialize txList  $\leftarrow \emptyset$ ,  $\vec{N} \leftarrow \varepsilon$ 
    $\triangleright$  Extract transaction timestamps
2: for  $i$  from 1 to  $\text{len}(\mathcal{C}) - K$  do
3:   for tag =  $(ct, \pi, \tau) \in \text{PB} \in \mathcal{B}_i$  do
4:     Set  $B = \{\mathcal{B}_j \mid i \leq j < i + K\}$ 
5:      $t \leftarrow \text{med}(\{t \mid (\text{tag}, t) \in \text{PB} \in B\} \cup \{+\infty \mid \text{tag} \notin \text{PB} \in B\})$ 
6:     if  $t \neq +\infty$  then Add (tag,  $t$ ) to txList
7:   end for
8: end for
    $\triangleright$  Construct transaction sequence
9: for  $i$  from 1 to  $\text{len}(\mathcal{C})$  do
10:   Set  $\vec{N}_i \leftarrow \varepsilon$ 
11:   if  $i \geq K + k$  then
12:     Let  $\tilde{\text{tag}}$  denote the set of transaction tag s.t. the  $K$ -window of tag ends in  $\mathcal{B}_{i-k}$ 
13:     Order  $\tilde{\text{tag}}$  non-decreasingly based on timestamp
14:     for tag  $\in \tilde{\text{tag}}$  do
15:       if there is tag' in block  $\mathcal{B}_{i-k+1}, \dots, \mathcal{B}_i$  s.t.  $\text{TS}(\text{tag}') < \text{TS}(\text{tag})$  then
16:          $\vec{N}_i \leftarrow \vec{N}_i \parallel \tilde{\text{tag}}_1 \parallel \dots \parallel \tilde{\text{tag}}_k \parallel \text{tag}$  where  $\tilde{\text{tag}}_1 \parallel \dots \parallel \tilde{\text{tag}}_k$  are tag that are in txList with
            $\text{TS}(\tilde{\text{tag}}_i) < \text{TS}(\text{tag})$  however not in  $\vec{N}$ 
17:       end if
18:     end for
19:   end if
20:    $\vec{N} \leftarrow \vec{N} \parallel \vec{N}_i$ 
21: end for
22: return  $\vec{N}$ 

```

Algorithm 7: *Extract transaction sequence from a chain.*

When the decryption of a transaction **tag** is available, that tag is replaced with the plain transaction. A state $\vec{\text{st}}$ is acquired by extracting a prefix of the transaction sequence $\vec{N}$ such that all transactions are decrypted.

Protocol ReadState(P, sid)

```

1: Invoke FetchInformation(sid, P) to receive the newest messages for this round
2:  $\vec{N} \leftarrow \text{ExtractTransactionSequence}(\mathcal{C}_{\text{loc}})$ 
3: for tag =  $(\pi, ct, \tau) \in \vec{N}$  do
4:   if there is a tx  $\in \mathcal{C}_{\text{loc}}$  such that tx is a signed decryption for  $ct$  then
5:     Replace tag with tx
6:   end if
7: end for
8: Extract the state  $\vec{\text{st}}$  from Blockify( $\vec{N}'$ ), where  $\vec{N}'$  is a prefix of  $\vec{N}$  up to a block such that all transactions are decrypted
9: Output (READ, sid,  $\vec{\text{st}}^{\lceil k}$ ) to  $\mathcal{Z}$ .

```

Protocol 8: *The read state procedure in $\Pi_{\text{Ledger}}^{\text{Fair}}$.*

6 Security Analysis

Overview of the security analysis. In the ideal world, the simulator $\mathcal{S}$ simulates the network functionality for a black-box adversary $\mathcal{A}$; $\mathcal{S}$ also simulates the ledger maintenance procedure of honest parties and jointly builds a blockchain with $\mathcal{A}$. When dummy parties issue new transactions, $\mathcal{S}$ generates a fake ciphertext of an all-zero string and send it to the simulated network. When an encrypted transaction is received from $\mathcal{A}$, $\mathcal{S}$ extracts the plain transaction by using Π_{NIZK} . Importantly, when the simulated blockchain has progressed such that $\mathcal{S}$ learns the transaction order, $\mathcal{S}$ proposes those transactions (using txid) to $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ and thus learn the honest transactions. $\mathcal{S}$ then programs the $\mathcal{G}_{\text{rpoRO}}$ (and hides this “isProgramed” information to $\mathcal{A}$) and equivocate the corresponding encrypted transactions to the real honest transaction.

We provide some more details on handling honest transactions. When the ledger $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ notifies $\mathcal{S}$ that a dummy honest party P submits a new transaction of length m with txid at time τ , $\mathcal{S}$ simulates the `IssueNewTransaction` procedure for P by associating a block $\mathcal{B}$ with this transaction, based on P ’s simulated local chain state (see `ISSUENEWTRANSACTION` in $\mathcal{S}_{\text{ledger}}$ Part 3). In order to learn the honest transactions, $\mathcal{S}$ waits until that in the simulated blockchain, all transactions that might precede the transaction with txid get settled. $\mathcal{S}$ proposes the sequence of transactions to $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ and thus learns the transaction $\mathbf{tx}$ that is associated with txid. Note that since the transaction decryption interval parameter Λ is set slightly longer than the time that it will get settled on the blockchain, $\mathcal{S}$ always has enough time to program $\mathcal{G}_{\text{rpoRO}}$ and equivocate the ciphertext of an all-zero string to $\mathbf{tx}$. We detail this part in `EXTENDLEDGERSTATE` in $\mathcal{S}_{\text{ledger}}$ Part 3.

The simulator terminates when certain bad event happens. Specifically, we define the following bad events `BAD-CP` (violation of common prefix), `BAD-CQ` (violation of chain quality), `BAD-CG` (violation of chain growth), `BAD-PROFILE` (violation of honest profile majority in any K consecutive blocks), `BAD-NIZK` (failure of the NIZK scheme), `BAD-DEC` (failure of transaction equivocation), `BAD-TICKET` (failure of selecting enclave public-key subset with at least one honest key). We prove that all these bad events happen with negligible probability with respect to the security parameter. Hence, the simulator can run the simulation for any polynomial time without abortion.

The simulator. We present the simulator used in the UC proof of $\Pi_{\text{Ledger}}^{\text{Fair}}$ that securely implements the ledger functionality $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$.

Simulator $\mathcal{S}_{\text{ledger}}$ (Part 1 - Main structure)

Overview:

- The simulator internally emulates all local UC functionalities by running the code (and keeping the state) of $\mathcal{G}_{\text{RO}}$, $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$, $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$ and $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$.
- The simulator mimics the execution of $\Pi_{\text{Ledger}}^{\text{Fair}}$ for each honest party U_p (including their state and the interaction with the hybrids).
- The simulator emulates a view towards the adversary $\mathcal{A}$ in a black-box way, i.e., by internally running adversary $\mathcal{A}$ and simulating his interaction with the protocol (and hybrids) as detailed below for each hybrid. To simplify the description, we assume $\mathcal{A}$ does not violate the requirements by the wrapper $\mathcal{W}(\mathcal{G}_{\text{RO}})$.
- For global functionalities $\mathcal{G}_{\text{RO}}$ and $\mathcal{G}_{\text{att}}$, the simulator simply relays the messages sent from $\mathcal{A}$ to the global functionalities (and returns the generated replies). Recall that the ideal world consists of the dummy parties, the ledger functionality, the clock, the global random oracles and the enclave.

Party sets: An honest miner P registered to $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ is assumed to be registered in all simulated

functionalities. Upon any activation, the simulator will query the current party set from the ledger (and simulate the corresponding message they send out to the network in the first maintain-ledger activation after registration), query all activations from honest parties $\vec{\mathcal{I}}_H^T$, and read the current clock value to learn the time. In particular, the simulator knows which parties are honest and synchronized and which parties are de-synchronized.

Messages from the clock: Upon receiving (CLOCK-UPDATE, sid_C, U_p) from $\mathcal{G}_{\text{Clock}}$, if U_p is an honest registered party, then remember that this party has received such a clock update (and the environment gets an activation). Otherwise, send (CLOCK-UPDATE, sid_C, U_p) to $\mathcal{A}$.

Messages from the ledger:

- Upon receiving (SUBMIT, m, txid, τ, P) from $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ where m denotes the transaction length, execute $\text{ISSUE_NEW_TRANSACTION}(P, \tau, m)$ and denote response by (π, ct, τ) . Forward (DIFFUSE, $\text{sid}, (\pi, ct, \tau)$) to the simulated network $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$ in the name of P . Output the answer of $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$ to the adversary.
- Upon receiving (MAINTAIN-LEDGER, $\text{sid}, \text{minerID}$) from $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$, extract from $\vec{\mathcal{I}}_H^T$ the party P that issued this query. If P has already completed its round-task, then ignore this request. Otherwise, execute $\text{SIMULATE_MINING}(P, \tau_{\mathcal{L}})$.

Messages from $\mathcal{G}_{\text{rpoRO}}$: $\mathcal{S}$ acts as a dummy adversary and forward all responses from $\mathcal{G}_{\text{rpoRO}}$ to its requestor except that when receiving a (IS-PROGRAMMED, m) request, return (IS-PROGRAMMED, false) if the m is previously programed by the simulator.

Simulator $\mathcal{S}_{\text{ledger}}$ (Part 2 - Black-Box Interaction)

Simulation of the Network $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ (over which chains are sent) towards $\mathcal{A}$:

- Upon receiving (DIFFUSE, $\text{sid}, (\mathcal{C}_{i_1}, U_{i_1}), \dots, (\mathcal{C}_{i_\ell}, U_{i_\ell})$) with a list of chains and corresponding parties from $\mathcal{A}$ (or on behalf some corrupted $P \in \mathcal{P}$), then do the following:
 1. Relay this input to the simulate network functionality and record its response to $\mathcal{A}$.
 2. Execute $\text{EXTEND_LEDGER_STATE}(\tau_{\mathcal{L}})$.
 3. Provide $\mathcal{A}$ with the recorded output of the simulated network.
- Upon receiving (DIFFUSE, $\text{sid}, \mathcal{C}$) from $\mathcal{A}$ on behalf of some *corrupted* party P , then do the following:
 1. Relay this input to the simulate network functionality and record its response to $\mathcal{A}$.
 2. Execute $\text{EXTEND_LEDGER_STATE}(\tau_{\mathcal{L}})$.
 3. Provide $\mathcal{A}$ with the recorded output of the simulated network.
- Upon receiving (FETCH, sid) from $\mathcal{A}$ on behalf some corrupted $P \in \mathcal{P}$ forward the request to the simulated $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ and return whatever is returned to $\mathcal{A}$.
- Upon receiving (DELAYS, $\text{sid}, (T_{\text{mid}_{i_1}}, \text{mid}_{i_1}), \dots, (T_{\text{mid}_{i_\ell}}, \text{mid}_{i_\ell})$) from $\mathcal{A}$ forward the request to the simulated $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ and record the answer to $\mathcal{A}$. Before giving this answer to $\mathcal{A}$, query the ledger state **state** and execute $\text{ADJUSTVIEW}(\text{state}, \tau_{\mathcal{L}})$.
- Upon receiving (SWAP, $\text{sid}, \text{mid}, \text{mid}'$) from $\mathcal{A}$ forward the request to the simulated $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ and record the answer to $\mathcal{A}$. Before giving this answer to $\mathcal{A}$, query the ledger state **state** and execute $\text{ADJUSTVIEW}(\text{state}, \tau_{\mathcal{L}})$.

Simulation of the Network $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$ (over which transactions are sent) towards $\mathcal{A}$:

- Upon receiving (DIFFUSE, $\text{sid}, \text{tx} = (\pi, ct, \tau)$) from $\mathcal{A}$ (or on behalf some corrupted $P \in \mathcal{P}$), then do the following:
 1. If τ is a valid ticket with respect to an existing block that only contains corrupted enclave public keys, **Abort** simulation: violation of good ticket (event BAD-TICKET)
 2. Extract tx from π using $\Pi_{\text{NIZK}}.\text{SimSetup}$, $\Pi_{\text{NIZK}}.\text{SimProve}$ and $\Pi_{\text{NIZK}}.\text{Extract}$. If extraction fails,

Abort simulation: violation of good NIZK (event BAD-NIZK)

3. Submit $\mathbf{tx}$ to the ledger on behalf of this corrupted party, and receive for the transaction id txid.
4. Forward the request to the internally simulated $\mathcal{F}_{\text{Diffuse}}^{\mathbf{tx}}$, which replies for each message with a message-ID mid.
5. Remember the association between mid and the corresponding txid.
6. Provide $\mathcal{A}$ with whatever the network outputs.
- Upon receiving (FETCH, sid) from $\mathcal{A}$ on behalf some corrupted $P \in \mathcal{P}$ forward the request to the simulated $\mathcal{F}_{\text{Diffuse}}^{\mathbf{tx}}$ and return whatever is returned to $\mathcal{A}$.
- Upon receiving (DELAYS, sid, $(T_{\text{mid}_{i_1}}, \text{mid}_{i_1}), \dots (T_{\text{mid}_{i_\ell}}, \text{mid}_{i_\ell})$) from $\mathcal{A}$ forward the request to the simulated $\mathcal{F}_{\text{Diffuse}}^{\mathbf{tx}}$ and return whatever is returned to $\mathcal{A}$.
- Upon receiving (SWAP, sid, mid, mid') from $\mathcal{A}$ forward the request to the simulated $\mathcal{F}_{\text{Diffuse}}^{\mathbf{tx}}$ and return whatever is returned to $\mathcal{A}$.

Simulation of the Network $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$ ($\mathcal{F}_{\text{Diffuse}}^{\text{pk}}$ resp.) (over which input blocks [enclave public keys resp.] are sent) towards $\mathcal{A}$:

- Upon receiving (DIFFUSE, sid, m) from $\mathcal{A}$ with an input block m on behalf some corrupted $P \in \mathcal{P}$, then do the following:
 1. Forward the request to the internally simulated $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$ ($\mathcal{F}_{\text{Diffuse}}^{\text{pk}}$ resp.), which replies for each message with a message-ID mid.
 2. Remember the association between mid and the corresponding input block.
 3. Provide $\mathcal{A}$ with whatever the network outputs.
- Upon receiving (FETCH, sid) from $\mathcal{A}$ on behalf some corrupted $P \in \mathcal{P}$ forward the request to the simulated $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$ ($\mathcal{F}_{\text{Diffuse}}^{\text{pk}}$ resp.) and return whatever is returned to $\mathcal{A}$.
- Upon receiving (DELAYS, sid, $(T_{\text{mid}_{i_1}}, \text{mid}_{i_1}), \dots (T_{\text{mid}_{i_\ell}}, \text{mid}_{i_\ell})$) from $\mathcal{A}$ forward the request to the simulated $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$ ($\mathcal{F}_{\text{Diffuse}}^{\text{pk}}$ resp.) and return whatever is returned to $\mathcal{A}$.
- Upon receiving (SWAP, sid, mid, mid') from $\mathcal{A}$ forward the request to the simulated $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$ ($\mathcal{F}_{\text{Diffuse}}^{\text{pk}}$ resp.) and return whatever is returned to $\mathcal{A}$.

Simulator $\mathcal{S}_{\text{ledger}}$ (Part 3 - Internal Procedures)

- SIMULATEMINING(P, τ) $\triangleleft$ Simulate the mining procedure of P in the protocol in round τ
- 1: Execute FetchInformation(P, sid)
 - 2: **if** Update $_{P, \tau}$ **then**
 - 3: Send (CLOCK-UPDATE, sid $_C$, P) to $\mathcal{A}$ if $\mathcal{S}_{\text{ledger}}$ has received such an input in round τ
 - 4: **else**
 - 5: Execute MiningProcedure(P, sid) and set Update $_{P, \tau} \leftarrow \text{true}$
 - 6: Before the activation goes to $\mathcal{A}$, execute EXTENDLEDGERSTATE(τ).
 - 7: **end if**
- EXTENDLEDGERSTATE(τ)
- 8: Let $\vec{N}$ be the longest state (extracted from $\mathcal{C}_{\text{loc}}$ by ExtractTransactionSequence) among all such states $\vec{N}_{U_p}$ where $U_p \in \mathcal{H}$
 - 9: Let $\vec{N}_{\mathcal{L}}$ denote the sequence reconstructed from state in $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$
 - 10: **if** $|\vec{N}_{\mathcal{L}}| > |\vec{N}^{[k]}|$ **then** Execute ADJUSTVIEW(state)
 - 11: **if** $\vec{N}_{\mathcal{L}}$ is not a prefix of $\vec{N}^{[k]}$ **then**
 - 12: **Abort** simulation: consistency violation. $\triangleleft$ Event BAD-CP
 - 13: **end if**

```

14: Define the difference  $\text{diff}$  to be the block sequence s.t.  $\vec{N}_{\mathcal{L}} \parallel \text{diff} = \vec{N}^{[k]}$ .
15: Parse  $\text{diff} = \text{diff}_1 \parallel \dots \parallel \text{diff}_n$ .
16: for  $j$  from 1 to  $n$  do
17:   Map each transaction  $\text{tx}$  in this block to its unique transaction ID  $\text{txid}$ .
18:   Let  $\text{list}_j = (\text{txid}_{j,1}, \dots, \text{txid}_{j,\ell_j})$  be the corresponding list for this block  $\text{diff}_j$ 
19:   if coinbase  $\text{txid}_{j,1}$  specifies a party honest at block creation time then
20:      $\text{hFlag} \leftarrow 1$ 
21:   else
22:      $\text{hFlag} \leftarrow 0$ 
23:   end if
24:   Output (NEXT-BLOCK,  $\text{list}_j$ ) to  $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$  and receive (NEXT-BLOCK, ok) as an immediate answer.
25: end for
26: if fraction of blocks with  $\text{hFlag} = 0$  in the recent  $k$  blocks  $> 1 - \mu_{\text{CQ}}$  then
27:   Abort simulation: chain quality violation  $\triangleleft$  Event BAD-CQ
28: else if state increases less than  $k$  blocks during the last  $k/\tau_{\text{CG}}$  rounds then
29:   Abort simulation: chain growth violation  $\triangleleft$  Event BAD-CG
30: else if honest profiles account for less than half in the last  $K$  blocks then
31:   Abort simulation: profile block violation  $\triangleleft$  Event BAD-PROFILE
32: end if
    $\triangleright$  Equivocate honest transactions.
33: Send (READ, sid) to  $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$  and receive state
34: for  $\text{tx} \in \text{state}$  and  $\text{txid}$  corresponds to a faked transaction and  $\text{equivocate}_{\text{tx}} = \text{false}$  do
35:   Let  $r_1, \dots, r_m$  denote the randomnesses used to generate  $ct_1, \dots, ct_m$  of  $\text{tx}$  (via  $\text{txid}$ )
36:   for  $i$  from 1 to  $m$  do
37:     Split  $r_i$  into  $|\text{tx}|$  pieces and for each piece  $r_{i,j}$  send (PROGRAM,  $r_{i,j}, h$ ) such that  $h \oplus ct_i =$ 
        $\text{tx}[j]$  to  $\mathcal{G}_{\text{rpoRO}}$ 
38:     if programming  $\mathcal{G}_{\text{rpoRO}}$  fails then
39:       Abort simulation: failure of decryption  $\triangleleft$  Event BAD-DEC
40:     end if
41:   end for
42:   Set  $\text{equivocate}_{\text{tx}} \leftarrow \text{true}$ 
43: end for
44: Execute ADJUSTVIEW(state)

   ADJUSTVIEW(state,  $\tau$ )  $\triangleleft$  Adjust the view of synchronized parties.
45: pointers  $\leftarrow \varepsilon$ 
46: for party  $U_p \in \mathcal{H}$  of round  $\tau$  do
47:   Execute ReadState( $U_p$ , sid) and let the chain's decoded state be  $\vec{\text{st}}_{U_p}$ 
48: end for
49: for each synchronized party  $U_p \in (\mathcal{H} \setminus \mathcal{P}_{DS})$  of round  $\tau$  do
50:   Determine the pointers  $\text{pt}_{U_p}$  s.t.  $\vec{\text{st}}_{U_p}^{[k]} = \text{state}|_{\text{pt}_{U_p}}$ 
51:   if such a point does not exist then
52:     return  $\triangleleft$  Call on invalid input or event BAD-CP occurred
53:   end if
54:   if Update $_{U_p, \tau}$  then pointers  $\leftarrow \text{pointers} \parallel (U_p, \text{pt}_{U_p})$ 
55: end for
56: Output (SET-SLACK, pointers) to  $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ 
    $\triangleright$  Adjust the view of desynchronized parties
57: pointers  $\leftarrow \varepsilon$ 
58: desyncStates  $\leftarrow \varepsilon$ 

```

```

59: for each desynchronized party  $U_p \in \mathcal{P}_{DS}$  of round  $\tau$  do
60:   if  $\text{Update}_{U_p, \tau} = \text{false}$  then
61:     Set  $\text{pt}_{U_p}$  to be  $|\vec{\text{st}}_{U_p}^{[k]}|$ 
62:      $\text{pointers} \leftarrow \text{pointers} \parallel (U_p, \text{pt}_{U_p})$ 
63:      $\text{desyncStates} \leftarrow \text{desyncStates} \parallel (U_p, \vec{\text{st}}_{U_p}^{[k]})$ 
64:   end if
65:   Output (SET-SLACK, pointers) to  $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ 
66:   Output (DESYNC-STATE, desyncStates) to  $\mathcal{G}_{\text{Ledger}}^{\text{Fair}}$ 
67: end for

  ISSUENewTransaction(P,  $\tau$ ,  $m$ )  $\triangleleft$  Issue dummy transactions
68: Let  $\mathcal{C}$  denote the simulated blockchain for party P
69:  $h \leftarrow \text{head}(\mathcal{C}^{[k]})$  and  $st \leftarrow$  blockchain state associated with  $h$ 
70:  $\mathcal{S}_{\text{pk}} \leftarrow$  all enclave public keys up to block with hash  $h$ 
71:  $\mathcal{S}_{\text{pk}}^\tau \leftarrow f_{\text{select}}(\mathcal{S}_{\text{pk}}, m, \tau)$  and  $m' \leftarrow |\mathcal{S}_{\text{pk}}^\tau|$ 
72: if  $\mathcal{S}_{\text{pk}}^\tau$  contains only corrupted enclave keys then
73:   Abort simulation: violation of good block ticket  $\triangleleft$  Event BAD-TICKET
74: end if
75: for  $i$  from 1 to  $m'$  do  $\triangleleft$  Encrypt an all-zero string
76:    $\text{pk}_i \leftarrow \mathcal{S}_{\text{pk}}^\tau[i], r_i \xleftarrow{\$} \{0, 1\}^\kappa$ 
77:    $ct_i \leftarrow \text{PKE.Enc}(\text{pk}_i, 0^m \parallel h; r_i)$  and remember  $r_i$  for equivocation later
78: end for
   $\triangleright$  Generate fake NIZK proof, telling an all-zero string is a valid transactino.
79:  $\pi \leftarrow \Pi_{\text{NIZK}}.\text{Prove}((h, st, (ct_1, \dots, ct_{m'}), (\text{pk}_1, \dots, \text{pk}_{m'})), (\text{tx}, (r_1, \dots, r_{m'})))$ 
80: return  $(\pi, ct, \tau)$ 

```

6.1 Blockchain Security Properties

In our protocol, a blockchain is built as an intermediate step to agree on transaction timestamp and provide long chains for decrypting transactions. We thus first focus on the good properties on the chain — namely, common prefix, chain growth and chain quality.

- **Common Prefix (CP); parameterized with $k \in \mathbb{N}^+$.** The chains $\mathcal{C}_1, \mathcal{C}_2$ possessed by two alert parties at the onset of the round $r_1 < r_2$ are such that $\mathcal{C}_1^{[k]} \preceq \mathcal{C}_2$, where $\mathcal{C}_1^{[k]}$ denotes the chain obtained by removing the last k blocks from $\mathcal{C}_1$, and $\preceq$ denotes the prefix relation.
- **Chain Growth (CG); parameterized with $\tau \in (0, 1]$ and $s \in \mathbb{N}$.** Consider a chain $\mathcal{C}$ possessed by an alert party at the onset of a round r . Let r_1 and r_2 be two previous rounds for which $r_1 + s \leq r_2 \leq r$. Then $|\mathcal{C}[r_1 : r_2]| \geq \tau \cdot s$ where τ is the speed coefficient.
- **Chain Quality (CQ); parameterized with $\mu \in (0, 1]$ and $s \in \mathbb{N}$.** Consider any portion of length at least s of the chain possessed by an alert party at the onset of a round; the ratio of blocks originating from alert parties is at least μ . We call μ the chain quality coefficient.

Note that our blockchain framework can be viewed as a “superset” of the Bitcoin backbone protocol, where they share the same chain structure (recall that all our revisions are regarding letting the chain accept new types of “profile blocks” and the transaction encryption scheme is blockchain-agnostic). Hence, following the similar arguments in [GKL15, PSs17, BMTZ17], the following Lemma 4 concluding that these properties hold throughout the entire execution except

with negligible probability is immediate. Note that to simplify the presentation, we omit the detailed parametrization of the protocol but only focus on the achieved properties. Specifically, common prefix is achieved by pruning last $k = \text{polylog}(\kappa)$ blocks, and the chain quality parameter $\mu > 0$, indicating that there is at least one honest block for every k consecutive blocks.

Lemma 4. *There exist protocol parametrizations such that the following properties hold throughout the an execution of $\Pi_{\text{Ledger}}^{\text{Fair}}$ for $L = \text{poly}(\kappa)$ rounds: (i) common prefix property with parameter $k = \Theta(\text{polylog}(\kappa))$ (ii) chain growth property with parameter τ_{CG} and s_{CG} ; and (iii) chain quality with parameter $\mu > 0$ and $s = k$, except with probability negligibly small in the security parameter κ .*

We next focus on the properties regarding profile blocks. We show that honest parties can always produce profile blocks that account for the majority, for any $K = \Theta(k)$ consecutive blocks on the blockchain. We provide a sketched proof using only the common prefix, chain growth and chain quality parameter; a more refined proof and parametrization can be found in, e.g., [PS17, KLS24].

Lemma 5 (Majority of honest profile blocks). *If common prefix, chain growth and chain quality properties (with the same parameters as those in Lemma 4) are not violated during the execution, then in an execution of $\Pi_{\text{Ledger}}^{\text{Fair}}$ over a lifetime of $L = \text{poly}(\kappa)$ rounds, for any segmentation of the blockchain of $K = \Theta(k)$ consecutive blocks, the majority of the profile blocks included are produced by honest parties.*

Proof (Sketch). Consider any consecutive K blocks $\mathcal{B}_i, \dots, \mathcal{B}_{i+K-1}$. Due to chain quality, there is at least one honest block $\mathcal{B}_h$ such that $i \leq h < i + k - 1$ and $\mathcal{B}_{h'}$ such that $i + K - k < h' \leq i + K - 1$. It suffices to show that the profile blocks produced by honest parties in the time interval between $\mathcal{B}_h$ and $\mathcal{B}_{h'}$ is larger than those produced by the corrupted parties in an interval between the time of t and t' where $t = \text{timestamp}(\mathcal{B}_i) - R$ and $t' = \text{timestamp}(\mathcal{B}_{i+K-1}) + k$. By setting $R = 3k$ and $K = \Theta(k)$ (the constants depends on the advantage of honest computational power compared with the corrupted parties, and the chain growth parameter τ_{CG}) we conclude the proof. $\square$

Given that for any sliding window of K blocks the majority of the profile blocks included are produced by honest parties, we consider a transaction $\mathbf{tx}$ enter the system at time t (when $\mathbf{tx}$ is issued by corrupted parties, let t denote the earliest time such that $\mathbf{tx}$ is learnt by at least one honest party). Every honest maintainer will receive $\mathbf{tx}$ at a time t' such that $t \leq t' < t + \Delta$, hence majority of the profile blocks report time in this interval — i.e., in the final ledger $\mathcal{L}$, the position of $\mathbf{tx}$ will be later than any transaction that enters the system at time before $t - \Delta$; meantime, $\mathbf{tx}$ is positioned at a place earlier than those transactions entering the system at time later than $t + \Delta$.

6.2 Composable Guarantees

Section 6.1 shows that when the simulator jointly builds the blockchain with the black-box adversary, bad events regarding the violation of good blockchain properties, namely BAD-CP, BAD-CQ, BAD-CG and BAD-PROFILE, happen with negligible probability. We next prove that the simulator $\mathcal{S}$ can simulate the transaction encryption mechanism well, by showing that all bad events BAD-TICKET, BAD-NIZK and BAD-DEC happen with negligible probability. Our proof makes use of the following large convergence bound.

Theorem 6 (Chernoff bounds). *Suppose $\{X_i : i \in [n]\}$ are mutually independent Boolean random variables, with $\Pr[X_i = 1] = p$, for all $i \in [n]$. Let $X = \sum_{i=1}^n X_i$ and $\mu = pn$. Then, for*

any $\delta \in (0, 1]$, it holds that

$$\Pr[X \leq (1 - \delta)\mu] \leq e^{-\delta^2\mu/2} \quad \text{and} \quad \Pr[X \geq (1 + \delta)\mu] \leq e^{-\delta^2\mu/3}.$$

Also, for all $t > 0$,

$$\Pr[X \geq \mu + t] \leq e^{-2t^2/n}.$$

First, we consider BAD-TICKET which implies that the adversary $\mathcal{A}$ can send an encrypted transaction such that all public keys used in the encryption are from enclaves controlled by the corrupted parties thus $\mathcal{A}$ can decide whether to open the transaction or not.

Lemma 7 (Good block tickets). *Consider an execution of $\Pi_{\text{Ledger}}^{\text{Fair}}$ over a lifetime of $L = \text{poly}(\kappa)$ rounds. The probability such that there exists a block ticket (h, τ) such that h equals the hash of a block in the settled part of the blockchain held by an honest party (at any time during the execution) is negligible with respect to the security parameter κ .*

Proof. Recall that the mining target T is appropriately parameterized such that the block generation rate $f < 1$ is a small constant, by applying Chernoff bound (Theorem 6), the number of blocks generated in $L = \text{poly}(\kappa)$ rounds is bounded by $(1 + \epsilon)f \cdot L$ except with probability $\exp(-\epsilon^2 f L / 3) = \exp(-\Omega(\text{poly}(\kappa)))$ which is negligibly small with respect to κ . I.e., consider n enclaves, at most $n \cdot \text{poly}(\kappa)$ block tickets will be associated with hash of a block in the settled blockchain.

Let $p \in (0, 1]$ denote the fraction of honest enclave public keys. We first consider a variant of Program 2 where for each hash h , the enclave returns a randomly selected value (and records this value for h for later queries), which applies a random subset selection on the set of enclave public-key set $\mathcal{S}_{pk}$ for each h . We show that by randomly select a subset of size $m = \text{polylog}(\kappa)$ in $\mathcal{S}_{pk}$ for $n \cdot \text{poly}(\kappa)$ times, the probability such that event E — there exists at least one selected subset such that all keys are from the enclaves controlled by corrupted parties — is negligible. For each random subset selection, the probability that no honest key is selected is bounded by $(1 - p)^m < \exp(-\Omega(\text{polylog} \kappa))$, by selecting $n \cdot \text{poly}(\kappa)$ we get $\Pr[E] = 1 - (1 - \exp(-\Omega(\text{polylog} \kappa)))^{n \cdot \text{poly}(\kappa)} = \exp(-\Omega(\text{polylog}(\kappa)) + \ln n)$ which is negligible in κ .

Now it suffices to show that by replacing the random number with $f_{\text{PRF}}(\text{key}, h)$, for each ticket there is still at least one honest key that gets selected. This can be proved by a reduction to the indistinguishability experiment of the underlying pseudorandom function f_{PRF} which we omit here (note that while each enclave possesses her own key, since the total number of the enclaves are polynomially bounded this does not hurt the argument). $\square$

Then we consider BAD-NIZK which implies that the simulator $\mathcal{S}$ cannot extract the encrypted transactions issued by the corrupted parties thus the simulation fails (note that it is not hard to see that the event such that honest parties fail to create a proof, or the corrupted parties generate a proof for a relation not in Equation (2) is negligible).

Lemma 8 (Good NIZK). *The probability that the simulator fails to extract the witness from a NIZK proof created by the black-box adversary is negligibly small in the security parameter κ .*

Proof. We prove this by a reduction to the security game of the special simulation-soundness (Game 1 in Definition 3), and show that if the simulator fails to extract, then the game returns **fail** with non-negligible probability, contradicting the fact that Π_{NIZK} is non-interactive special simulation-soundness.

The reduction in general works as follows. When the simulator $\mathcal{S}$ is to generate a NIZK proof for dummy honest parties, $\mathcal{S}$ forwards the query (Prove, x, w) as that in Game 1; when receiving a

proof (x, π) from the black-box adversary, $\mathcal{S}$ forwards $(\text{Challenge}, x, \pi)$ and extracts the witness. If this extraction fails and simulator aborts, then in Game 1 it must be the case that after running the Extract algorithm and get witness w it holds that $R(x, w) = 0$. I.e., if the simulator aborts for the failed extraction with non-negligible probability, then in Game 1 it also returns **Fail** with non-negligible probability. $\square$

Recall that in Lemma 3, the black-box adversary $\mathcal{A}$ cannot distinguish if a plaintext of PKE is decrypted by Dec or Dec*, finally we consider the event BAD-DEC which implies that the simulator fails to equivocate a faked ciphertext to its corresponding honest transaction.

Lemma 9 (Good decryption). *The probability such that when $\mathcal{A}$ queries $\mathcal{G}_{\text{att}}$ a faked ciphertext ct of an all-zero string corresponding to transaction $\mathbf{tx}$ and $\mathcal{G}_{\text{att}}$ returns a response other than $(\text{TX-DEC}, (\mathbf{tx}, ct))$ is negligible with respect to the security parameter κ .*

Proof. The event of BAD-DEC happens, because either $\mathcal{S}$ has no time to program the random oracle, or the programming fails.

We first consider the case that $\mathcal{S}$ has no time to program. Note that since the decryption parameter is set as $\Lambda > 4k + K$, for an honest transaction $\mathbf{tx}$ that associates with a block with height ℓ , there will exist an honest block with height at most $\ell + 2k$ (due to chain quality) such that it contains the profile blocks with $\mathbf{tx}$. I.e., when the blockchain has progressed to length $\ell + 4k + K$, the timestamp of $\mathbf{tx}$ get settled, and the same holds for all transactions that enters the system before $\mathbf{tx}$. Hence $\mathcal{S}$ can propose a block containing $\mathbf{tx}$ and learn its plaintext before the chain grows to length $\ell + \Lambda > \ell + 4k + K$; i.e., when the simulator program all ciphertext for $\mathbf{tx}$ the TX-DEC interface will never try to use Dec* to decrypt ct of $\mathbf{tx}$.

Regarding the event that programming $\mathcal{G}_{\text{rpoRO}}$ fails, note that this only happens when the point x to program has already been queried before. Note that for any point x to program, there exists a (piece of) ciphertext ct with respect to $\mathbf{pk} = (f, b)$, such that $ct = f(x)$. If the event that any programming on point x fails, then it implies $\Pr_{x \in \{0,1\}^\kappa} [\mathcal{A}(f(x)) \in f^{-1}(f(x))]$ is non-negligible, contradicting the fact that f is a secure one-way function. $\square$

We conclude that our protocol $\Pi_{\text{Ledge}}^{\text{Fair}}$, when appropriately parameterized, securely realizes $\mathcal{G}_{\text{Ledge}}^{\text{Fair}}$ in an $(\mathcal{G}_{\text{Clock}}, \mathcal{W}(\mathcal{G}_{\text{RO}}), \mathcal{F}_{\text{Diffuse}}, \mathcal{G}_{\text{att}}, \mathcal{G}_{\text{rpoRO}})$ -hybrid environment.

Theorem 10. *Let Δ denote the network delay, τ_{CG} the chain-growth coefficient, k the common prefix parameter, K the length of profile block window and PKE, Λ the parameter of Program 1 where PKE is as specified in Algorithm 4. Assuming honest majority in terms of computational power and honest parties controlling a constant fraction of enclave public keys, there exists protocol parametrization ($k = \Theta(\Delta \log^2 \kappa)$, $K = \Theta(k)$ and $\Lambda = K + 5k$, $h/n = \Omega(\log^2 \kappa)$) such that Protocol $\Pi_{\text{Ledge}}^{\text{Fair}}$ UC-realizes $\mathcal{G}_{\text{Ledge}}^{\text{Fair}}$ parameterized with Δ -ApproxSenderOrder, with `windowSize` = k , `Delay` = 2Δ and `waitTime` = $(4k + K)/\tau_{\text{CG}}$, in the $(\mathcal{G}_{\text{Clock}}, \mathcal{W}(\mathcal{G}_{\text{RO}}), \mathcal{F}_{\text{Diffuse}}, \mathcal{G}_{\text{att}}, \mathcal{G}_{\text{rpoRO}})$ -hybrid environment.*

Proof (Sketch). We prove the theorem by providing a simulator $\mathcal{S}_{\text{ledger}}$ in the ideal world such that the protocol execution in the $(\mathcal{G}_{\text{Clock}}, \mathcal{W}(\mathcal{G}_{\text{RO}}), \mathcal{F}_{\text{Diffuse}}, \mathcal{G}_{\text{att}}, \mathcal{G}_{\text{rpoRO}})$ -hybrid world is indistinguishable from the ideal-world execution with the ledger functionality and the simulator. This simulation can be done perfectly, as the only events that prevent a successful simulation are those defined by BAD-CP, BAD-CQ, BAD-CG, BAD-PROFILE, BAD-NIZK and BAD-DEC, which we have been proving in the previous Lemmas 4, 5 and 7 to 9 that happens with negligible probability. $\square$

Acknowledgements

Yu Shen’s research has been supported by Input Output (iohk.io) through their funding of the University of Edinburgh Blockchain Technology Lab.

References

- [BCH⁺20] Christian Badertscher, Ran Canetti, Julia Hesse, Björn Tackmann, and Vassilis Zikas. Universal composition with global subroutines: Capturing global setup within plain UC. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part III*, volume 12552 of *Lecture Notes in Computer Science*, pages 1–30, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-64381-2_1.
- [BGG⁺20] Fabrice Benhamouda, Craig Gentry, Sergey Gorbunov, Shai Halevi, Hugo Krawczyk, Chengyu Lin, Tal Rabin, and Leonid Reyzin. Can a public blockchain keep a secret? In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part I*, volume 12550 of *Lecture Notes in Computer Science*, pages 260–290, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-64375-1_10.
- [BGK⁺18] Christian Badertscher, Peter Gazi, Aggelos Kiayias, Alexander Russell, and Vassilis Zikas. Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 913–930, Toronto, ON, Canada, October 15–19, 2018. ACM Press. <https://doi.org/10.1145/3243734.3243848>.
- [BMTZ17] Christian Badertscher, Ueli Maurer, Daniel Tschudi, and Vassilis Zikas. Bitcoin as a transaction ledger: A composable treatment. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 324–356, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-63688-7_11.
- [Can00] Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13(1):143–202, January 2000. <https://doi.org/10.1007/s001459910006>.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science*, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press. <https://doi.org/10.1109/SFCS.2001.959888>.
- [CD16] Victor Costan and Srinivas Devadas. Intel SGX explained. Cryptology ePrint Archive, Report 2016/086, 2016. URL: <https://eprint.iacr.org/2016/086>.
- [CDG⁺18] Jan Camenisch, Manu Drijvers, Tommaso Gagliardoni, Anja Lehmann, and Gregory Neven. The wonderful world of global random oracles. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 280–312, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-78381-9_11.
- [CKPS01] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. Secure and efficient asynchronous broadcast protocols. In Joe Kilian, editor, *Advances in Cryptology – CRYPTO 2001*, volume 2139 of *Lecture Notes in Computer Science*, pages 524–541, Santa Barbara, CA, USA, August 19–23, 2001. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/3-540-44647-8_31.
- [CMSZ22] Christian Cachin, Jovana Micic, Nathalie Steinhauer, and Luca Zanolini. Quick order fairness. In Ittay Eyal and Juan A. Garay, editors, *FC 2022: 26th International Conference on Financial Cryptography and Data Security*, volume 13411 of *Lecture Notes in Computer Science*, pages 316–333, Grenada, May 2–6, 2022. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-031-18283-9_15.

- [DGK⁺20] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE Symposium on Security and Privacy*, pages 910–927, San Francisco, CA, USA, May 18–21, 2020. IEEE Computer Society Press. <https://doi.org/10.1109/SP40000.2020.00040>.
- [DM16] George Danezis and Sarah Meiklejohn. Centrally banked cryptocurrencies. In *ISOC Network and Distributed System Security Symposium – NDSS 2016*, San Diego, CA, USA, February 21–24, 2016. The Internet Society. <https://doi.org/10.14722/ndss.2016.23187>.
- [FGKR20] Matthias Fitzi, Peter Gazi, Aggelos Kiayias, and Alexander Russell. Ledger combiners for fast settlement. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part I*, volume 12550 of *Lecture Notes in Computer Science*, pages 322–352, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-64375-1_12.
- [GKL15] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol: Analysis and applications. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 281–310, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-662-46803-6_10.
- [GKL17] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol with chains of variable difficulty. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 291–323, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-63688-7_10.
- [GKM⁺22] Rachid Guerraoui, Petr Kuznetsov, Matteo Monti, Matej Pavlovic, and Dragos-Adrian Seredinschi. The consensus number of a cryptocurrency. *Distributed Computing*, 35(1):1–15, Feb 2022. <https://doi.org/10.1007/s00446-021-00399-2>.
- [KDK22] Mahimna Kelkar, Soubhik Deb, and Sreeram Kannan. Order-fair consensus in the permissionless setting. In Jason Paul Cruz and Naoto Yanai, editors, *APKC ’22: Proceedings of the 9th ACM on ASIA Public-Key Cryptography Workshop, APKC@AsiaCCS 2022, Nagasaki, Japan, 30 May 2022*, pages 3–14. ACM, 2022. <https://doi.org/10.1145/3494105.3526239>.
- [KLS24] Aggelos Kiayias, Nikos Leonardos, and Yu Shen. Ordering transactions with bounded unfairness: Definitions, complexity and constructions. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part III*, volume 14653 of *Lecture Notes in Computer Science*, pages 34–63, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-031-58734-4_2.
- [KMTZ13] Jonathan Katz, Ueli Maurer, Björn Tackmann, and Vassilis Zikas. Universally composable synchronous computation. In Amit Sahai, editor, *TCC 2013: 10th Theory of Cryptography Conference*, volume 7785 of *Lecture Notes in Computer Science*, pages 477–498, Tokyo, Japan, March 3–6, 2013. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-642-36594-2_27.
- [KO04] Jonathan Katz and Rafail Ostrovsky. Round-optimal secure two-party computation. In Matthew Franklin, editor, *Advances in Cryptology – CRYPTO 2004*, volume 3152 of *Lecture Notes in Computer Science*, pages 335–354, Santa Barbara, CA, USA, August 15–19, 2004. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-540-28628-8_21.
- [Kur20] Klaus Kursawe. Wendy, the good little fairness widget: Achieving order fairness for blockchains. In *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies, AFT ’20*, pages 25–36, New York, NY, USA, 2020. Association for Computing Machinery. <https://doi.org/10.1145/3419614.3423263>.

- [KZGJ20] Mahimna Kelkar, Fan Zhang, Steven Goldfeder, and Ari Juels. Order-fairness for byzantine consensus. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part III*, volume 12172 of *Lecture Notes in Computer Science*, pages 451–480, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-56877-1_16.
- [LR22] Anna Lysyanskaya and Leah Namisa Rosenbloom. Universally composable Σ -protocols in the global random-oracle model. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022: 20th Theory of Cryptography Conference, Part I*, volume 13747 of *Lecture Notes in Computer Science*, pages 203–233, Chicago, IL, USA, November 7–10, 2022. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-031-22318-1_8.
- [MS23] Dahlia Malkhi and Pawel Szalachowski. Maximal Extractable Value (MEV) Protection on a DAG. In Yackolley Amoussou-Guenou, Aggelos Kiayias, and Marianne Verdier, editors, *4th International Conference on Blockchain Economics, Security and Protocols (Tokenomics 2022)*, volume 110 of *Open Access Series in Informatics (OASICs)*, pages 6:1–6:17, Dagstuhl, Germany, 2023. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. <https://doi.org/10.4230/OASICs.Tokenomics.2022.6>.
- [Nak08] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, Dec 2008. URL: <https://bitcoin.org/bitcoin.pdf>.
- [PS17] Rafael Pass and Elaine Shi. FruitChains: A fair blockchain. In Elad Michael Schiller and Alexander A. Schwarzmann, editors, *36th ACM Symposium Annual on Principles of Distributed Computing*, pages 315–324, Washington, DC, USA, July 25–27, 2017. Association for Computing Machinery. <https://doi.org/10.1145/3087801.3087809>.
- [PSL80] Marshall C. Pease, Robert E. Shostak, and Leslie Lamport. Reaching agreement in the presence of faults. *J. ACM*, 27(2):228–234, apr 1980. <https://doi.org/10.1145/322186.322188>.
- [PSs17] Rafael Pass, Lior Seeman, and abhi shelat. Analysis of the blockchain protocol in asynchronous networks. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part II*, volume 10211 of *Lecture Notes in Computer Science*, pages 643–673, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-56614-6_22.
- [PST17] Rafael Pass, Elaine Shi, and Florian Tramèr. Formal abstractions for attested execution secure processors. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part I*, volume 10210 of *Lecture Notes in Computer Science*, pages 260–289, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-56620-7_10.
- [RB94] Michael K. Reiter and Kenneth P. Birman. How to securely replicate services. *ACM Transactions on Programming Languages and Systems*, 16(3):986–1009, 1994.
- [Sch90] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: a tutorial. *ACM Comput. Surv.*, 22(4):299–319, dec 1990. <https://doi.org/10.1145/98163.98167>.
- [ZSC⁺20] Yunhao Zhang, Srinath Setty, Qi Chen, Lidong Zhou, and Lorenzo Alvisi. Byzantine ordered consensus without byzantine oligarchy. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, USA, 2020. USENIX Association.

A Preliminaries (Cont’d)

Global clock. We model synchronous processors using $\mathcal{G}_{\text{Clock}}$ (cf. [KMTZ13]). At a high level, $\mathcal{G}_{\text{Clock}}$ maintains a round variable τ for each session, and the round forwards only when all registered parties send $\mathcal{G}_{\text{Clock}}$ the CLOCK-UPDATE command. The current time can be checked by sending

a CLOCK-READ command. Whenever a party who has finished the computation in a round is activated, she checks if the time from $\mathcal{G}_{\text{Clock}}$ has been forwarded; if not, she does nothing and wait for the next activation.

Functionality $\mathcal{G}_{\text{Clock}}$

This functionality maintains state variables as follows.

State Variable	Description
$\mathcal{P} \leftarrow \emptyset$	The set of registered parties $\mathbf{P} = (\text{pid}, \text{sid})$.
$F \leftarrow \emptyset$	The set of registered functionalities (together with their session identifier).
$\tau_{\text{sid}} \leftarrow 0$	The clock variable for session sid .
$d_{\mathbf{P}} \leftarrow 0$	The clock-update variable for $\mathbf{P} = (\text{pid}, \text{sid}) \in \mathcal{P}$.
$d(\mathcal{F}, \text{sid}) \leftarrow 0$	The clock-update variable for $(\mathcal{F}, \text{sid}) \in F$.

- Upon receiving (CLOCK-UPDATE, sid_C) from some party $\mathbf{P} \in \mathcal{P}$ set $d_{\mathbf{P}} \leftarrow 1$; execute *Round-Update* and forward (CLOCK-UPDATE, $\text{sid}_C, \mathbf{P}$) to $\mathcal{A}$.
- Upon receiving (CLOCK-UPDATE, sid_C) from some functionality $\mathcal{F}$ in a session sid such that $(\mathcal{F}, \text{sid}) \in F$ set $d(\mathcal{F}, \text{sid}) \leftarrow 1$, execute *Round-Update* and return (CLOCK-UPDATE, $\text{sid}_C, \mathcal{F}$) to this instance of $\mathcal{F}$.
- Upon receiving (CLOCK-READ, sid_C) from any participant (including the environment on behalf of a party, the adversary, or any ideal—shared or local—functionality) return (CLOCK-READ, $\text{sid}_C, \tau_{\text{sid}}$) to the requestor (where sid is the sid of the calling instance).

Procedure Round-Update: For each session sid do: If $d(\mathcal{F}, \text{sid}) = 1$ for all $\mathcal{F} \in F$ and $d_{\mathbf{P}} = 1$ for all honest partyset $\mathbf{P} = (\cdot, \text{sid}) \in \mathcal{P}$, then set $\tau_{\text{sid}} \leftarrow \tau_{\text{sid}} + 1$ and reset $d(\mathcal{F}, \text{sid}) \leftarrow 0$ and $d_{\mathbf{P}} \leftarrow 0$ for all partyset $\mathbf{P} = (\cdot, \text{sid}) \in \mathcal{P}$.

Functionality 4: *The global clock.*

Diffuse functionalities. We model the Δ -bounded delay network with $\mathcal{F}_{\text{Diffuse}}$ [BMTZ17]. Note that once a corrupted message reaches at least one honest party at round r , $\mathcal{F}_{\text{Diffuse}}$ guarantees to deliver this message to all honest parties before round $r + \Delta$ (i.e., honest parties keep “echoing” messages). By convention, different types of messages are diffused by different functionalities, and we write $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$, $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$, $\mathcal{F}_{\text{Diffuse}}^{\text{pb}}$, $\mathcal{F}_{\text{Diffuse}}^{\text{pk}}$ to denote the network for chains, transactions, profile blocks and enclave public keys respectively.

Functionality $\mathcal{F}_{\text{Diffuse}}$

The functionality is parameterized by the network delay Δ .

State Variable	Description
$\mathcal{P} \leftarrow \emptyset$	The set of registered parties.
$\vec{M} \leftarrow []$	A dynamically updatable list of quadruples $(m, \text{mid}, D_{\text{mid}}, P)$ where D_{mid} denotes the fetch counter.

Network capabilities:

- Upon receiving (DIFFUSE, sid, m) from some $P_s \in \mathcal{P}$, where $\mathcal{P} = \{P_1, \dots, P_n\}$ denotes the current party set, do:
 1. Choose n new unique message-IDs $\text{mid}_1, \dots, \text{mid}_n$.
 2. Initialize $2n$ new variables $D_{\text{mid}_1} := D_{\text{mid}_1}^{MAX} \dots := D_{\text{mid}_n} := D_{\text{mid}_n}^{MAX} := 1$ and a per message-delay $\Delta_{\text{mid}_i} = \Delta$ for $i \in [n]$.
 3. Set $\vec{M} := \vec{M} \parallel (m, \text{mid}_1, D_{\text{mid}_1}, P_1) \parallel \dots \parallel (m, \text{mid}_n, D_{\text{mid}_n}, P_n)$.
 4. Send (DIFFUSE, sid, $m, P_s, (P_1, \text{mid}_1), \dots, (P_n, \text{mid}_n)$) to the adversary.
- Upon receiving (FETCH, sid) from $P \in \mathcal{P}$, or from $\mathcal{A}$ on behalf of a corrupted party P :
 1. For all tuples $(m, \text{mid}, D_{\text{mid}}, P) \in \vec{M}$, set $D_{\text{mid}} := D_{\text{mid}} - 1$.
 2. Let $\vec{M}_0^P$ denote the subvector $\vec{M}$ including all tuples of the form $(m, \text{mid}, D_{\text{mid}}, P)$ with $D_{\text{mid}} \leq 0$ (in the same order as they appear in $\vec{M}$). Delete all entries in $\vec{M}_0^P$ from $\vec{M}$ and in case some $(m, \text{mid}, D_{\text{mid}}, P)$ is in $\vec{M}_0^P$, where P is honest, set $\Delta_{\text{mid}'} = \Delta$ for any $(m, \text{mid}', D_{\text{mid}'}, (\cdot, \text{sid}))$ in $\vec{M}$ and replace this record by $(m, \text{mid}', \min\{D_{\text{mid}'}, \Delta\}, P')$.
 3. Output $\vec{M}_0^P$ to P (if P is corrupted, send $\vec{M}_0^P$ to $\mathcal{A}$).

Additional adversarial capabilities:

- Upon receiving (DIFFUSE, sid, m) from some corrupted $P_s \in \mathcal{P}$ (or from $\mathcal{A}$ on behalf of P_s if corrupted), execute it the same way as an honest-sender diffuse, with the only difference that $\Delta_{\text{mid}_i} = \infty$.
- Upon receiving (DELAYS, sid, $(T_{\text{mid}_{i_1}}, \text{mid}_{i_1}), \dots, (T_{\text{mid}_{i_\ell}}, \text{mid}_{i_\ell})$) from the adversary do the following for each pair $(T_{\text{mid}_{i_j}}, \text{mid}_{i_j})$: if $D_{\text{mid}_{i_j}}^{MAX} + T_{\text{mid}_{i_j}} \leq \Delta_{\text{mid}_{i_j}}$ and mid_{i_j} is a message-ID of receiver $P = (\cdot, \text{sid})$ registered in the current $\vec{M}$, set $D_{\text{mid}_{i_j}} := D_{\text{mid}_{i_j}} + T_{\text{mid}_{i_j}}$ and set $D_{\text{mid}_{i_j}}^{MAX} := D_{\text{mid}_{i_j}}^{MAX} + T_{\text{mid}_{i_j}}$; otherwise, ignore this pair.
- Upon receiving (SWAP, sid, mid, mid') from the adversary, if mid and mid' are message-IDs registered in the current $\vec{M}$, then swap the triples $(m, \text{mid}, D_{\text{mid}}, (\cdot, \text{sid}))$ and $(m, \text{mid}', D_{\text{mid}'}, (\cdot, \text{sid}))$ in $\vec{M}$. Return (SWAP, sid) to the adversary.
- Upon receiving (GET-REG, sid) from $\mathcal{A}$, return the response (GET-REG, sid, $\mathcal{P}$) to $\mathcal{A}$.

Functionality 5: *The diffusion network.*

Random oracle for PoW and its wrapper. Our random oracle for generating PoW follows that in [BMTZ17] except that now it is a shared functionality.

Functionality $\mathcal{G}_{\text{RO}}$

The functionality is parameterized by a security parameter κ .

State Variable	Description
----------------	-------------

$\mathcal{P} \leftarrow \emptyset$	The set of registered parties.
$H \leftarrow \emptyset$	A dynamically updatable function table where $H[x] = \perp$ denotes the fact that no pair of the form $(x, \cdot)$ is in H .

- **Eval.** Upon receiving $(\text{EVAL}, \text{sid}_{RO}, x)$ from some party $P \in \mathcal{P}$ (or from $\mathcal{A}$ on behalf of a corrupted P), do the following:
 1. If $H[x] = \perp$ sample a value y uniformly at random from $\{0, 1\}^\kappa$ and set $H[x] \leftarrow y$.
 2. Return $(\text{EVAL}, \text{sid}, x, H[x])$ to the requestor.

Functionality 6: *The global random oracle.*

In order to limit the adversary on making a certain number of queries per round, we adopt a functionality wrapper [BMTZ17] that wraps the corresponding resource to capture such restrictions. Note that since now $\mathcal{G}_{RO}$ is shared among different sessions, the adversary can ask the environment to make queries on his behalf. Hence, our wrapper $\mathcal{W}(\mathcal{G}_{RO})$ also puts restrictions on queries that are made by the environment.

Functionality $\mathcal{W}(\mathcal{G}_{RO})$

This functionality maintains state variables as follows.

State Variable	Description
$\mathcal{P} \leftarrow \emptyset$	The set of registered parties; the current set of corrupted parties is denoted by $\mathcal{P}'$.
$\tau \leftarrow 0$	The (real-time) clock tick counter.
h_τ	An upper bound which restricts the $\mathcal{F}$ -evaluations of all alert parties at time τ .
$q_{\mathcal{H}}, q_{\mathcal{A}} \leftarrow 0$	The alert/adversary evaluation counter.

Relaying inputs to the random oracle:

- Upon receiving $(\text{EVAL}, \text{sid}, x)$ from $\mathcal{A}$ on behalf of a corrupted party $P \in \mathcal{P}'$ or a de-synchronized party P , first execute *Round Reset*, then do the following.
 1. Set $q_{\mathcal{A}} \leftarrow q_{\mathcal{A}} + 1$.
 2. **If** $q_{\mathcal{A}} \leq h_\tau$ **then** forward the request to $\mathcal{G}_{RO}$ and return to $\mathcal{A}$ whatever $\mathcal{G}_{RO}$ returns.
- Upon receiving $(\text{EVAL}, \text{sid}, x)$ from an alert party P , first execute *Round Reset*, then do the following.
 1. Set $q_{\mathcal{H}} \leftarrow q_{\mathcal{H}} + 1$.
 2. **If** $q_{\mathcal{H}} \leq h_\tau$ **then** forward the request to $\mathcal{G}_{RO}$ and return to P whatever $\mathcal{G}_{RO}$ returns.
 3. **If** $q_{\mathcal{H}} \geq h_\tau$ **then** send $(\text{CLOCK-UPDATE}, \text{sid}_C)$ to $\mathcal{G}_{\text{Clock}}$.

Corruption handling:

- Upon receiving $(\text{CORRUPT}, \text{sid}, P)$ from the adversary, set $\mathcal{P}' \leftarrow \mathcal{P}' \cup P$.

Procedure Round-Reset: Send $(\text{CLOCK-READ}, \text{sid}_C)$ to $\mathcal{G}_{\text{Clock}}$ and receive $(\text{CLOCK-READ}, \text{sid}_C, \tau')$ from $\mathcal{G}_{\text{Clock}}$. If $|\tau - \tau'| > 0$, then set $q_{\mathcal{H}}, q_{\mathcal{A}} \leftarrow 0$ and $\tau \leftarrow \tau'$.

Functionality 7: *The random oracle wrapper.*

Global random oracle. The restricted programmable and observable global random oracle $\mathcal{G}_{\text{rpoRO}}$ follows the modelling in [CDG⁺18]. We allow the adversary to observe and program the GRO; meantime, every party (in the same session) can check if a point is programmed by calling the “IsProgrammed” interface.

Functionality $\mathcal{G}_{\text{rpoRO}}$

The functionality is parameterized by the security parameter κ . It maintains a dynamically updatable list H and prog . Initially, $H = \text{prog} = \emptyset$.

- **Eval.** Upon receiving (EVAL, m) from a party (P, sid) or from the adversary, do the following:
 1. If $H[m] = \perp$, sample a value h uniformly at random from $\{0, 1\}^\kappa$ and set $H[m] \leftarrow h$.
 2. Parse m as (s, m') .
 3. If the query is made by the adversary, or if $s \neq \text{sid}$, then add (s, m', h) to the (initially empty) list of illegitimate queries $\mathcal{Q}_s$.
 4. Output (EVAL, m, h) to the requestor.
- **Observe.** Upon receiving $(\text{OBSERVE}, \text{sid})$ from the adversary: If $\mathcal{Q}_{\text{sid}}$ does not exist, set $\mathcal{Q}_{\text{sid}} = \emptyset$. Output $(\text{OBSERVE}, \mathcal{Q}_{\text{sid}})$ to the adversary.
- **Program.** Upon receiving $(\text{PROGRAM}, m, h)$ with $h \in \{0, 1\}^\kappa$ from the adversary, do the following:
 1. If $\exists h' \in \{0, 1\}^\kappa$ such that $H[m] = h'$ and $h \neq h'$, ignore this input.
 2. Set $H[m] \leftarrow h$ and $\text{prog} \leftarrow \text{prog} \cup \{m\}$.
 3. Output $(\text{PROGRAM}, \text{ok})$ to the adversary.
- **IsProgrammed.** Upon receiving $(\text{IS-PROGRAMMED}, m)$ from a party (P, sid) or from the adversary, do the following:
 1. If the input was given by (P, sid) , parse m as (s, m') . If $s \neq \text{sid}$, ignore this input.
 2. Set $b \leftarrow m \in \text{prog}$ and output $(\text{IS-PROGRAMMED}, b)$ to the requestor.

Functionality 8: *The restricted global random oracle with programmability and observability.*

Non-interactive zero knowledge scheme Π_{NIZK} . The NIZKPoK scheme that we use in this paper are from [LR22], specifically a non-interactive, straight-line extractable (NISLE) proof system satisfying the following three properties.

Definition 1 (Overwhelming Completeness). A NISLE proof system $\Pi_R^{\text{SLC}} = (\text{Setup}^H, \text{Prove}^H, \text{Verify}^H, \text{SimSetup}, \text{SimProve}, \text{Extract})$ for relation R in the random-oracle model has the overwhelming completeness property if for any security parameter λ , any random oracle H , any $(x, w) \in R$, and any proof $\pi \leftarrow \Pi_R^{\text{SLC}}.\text{Prove}^H(x, w)$, $\Pr[\Pi_R^{\text{SLC}}.\text{Verify}^H(x, \pi) = 1] \geq 1 - \text{negl}(\lambda)$.

Definition 2 (Non-Interactive Multiple SHVZK). A NISLE proof system $\Pi_R^{\text{SLC}} = (\text{Setup}^H, \text{Prove}^H, \text{Verify}^H, \text{SimSetup}, \text{SimProve}, \text{Extract})$ for relation R in the random-oracle model has the non-interactive multiple special honest-verifier zero-knowledge (NIM-SHVZK) property if for any security parameter λ , any random oracle H , any PPT adversary $\mathcal{A}$, and a bit $b \xleftarrow{\$} \{0, 1\}$, there exist some negligible function negl such that $\Pr[b' = b] \leq 1/2 + \text{negl}(\lambda)$, where b' is the result of running the game $\text{NIM-SHVZK}_{\mathcal{A}, \Pi_R^{\text{SLC}}}^{H, *}(1^\lambda, b)$. We say $\mathcal{A}$ wins the NIM-SHVZK game if $\Pr[b' = b] > 1/2 + \text{negl}(\lambda)$.

Definition 3 (Non-Interactive Special Simulation-Soundness). A NISLE proof system $\Pi_R^{\text{SLC}} = (\text{Setup}^H, \text{Prove}^H, \text{Verify}^H, \text{SimSetup}, \text{SimProve}, \text{Extract})$ for relation R in the random oracle model has the non-interactive special simulation-soundness property if for any security parameter λ , any random oracle H , any PPT adversary $\mathcal{A}$, there exist some negligible function negl such that $\Pr[\text{Fail} \leftarrow \text{NIM-SSS}_{\mathcal{A}, \Pi_R^{\text{SLC}}}^{H^*, \text{Prog}}(1^\lambda)] \leq \text{negl}(\lambda)$.

For completeness, we also describe the security game $\text{NIM-SSS}_{\mathcal{A}, \Pi_R^{\text{SLC}}}^{H^*, \text{Prog}}(1^\lambda)$ related to the non-interactive special simulation-soundness property of the NIZK protocol where $\mathcal{Q}_{\mathcal{A}}$ are $\mathcal{A}$'s queries to the RO (details see [LR22]).

Game $\text{NIM-SSS}_{\mathcal{A}, \Pi_R^{\text{SLC}}}^{H^*, \text{Prog}}(1^\lambda)$

```

1:  $L \leftarrow \perp$ 
2:  $\text{ppm}, z \leftarrow \Pi_R^{\text{SLC}}.\text{SimSetup}^{\text{Prog}_L}(1^\lambda)$ 
3:  $\text{st} \leftarrow \mathcal{A}^{H_L}(1^\lambda, \text{ppm})$ 
4:  $\text{pflist}, \text{Response} \leftarrow \perp$ 
5: while  $\text{st} \neq \perp$  do
6:    $(\text{Query}, \mathcal{Q}_{\mathcal{A}}, \text{st}) \leftarrow \mathcal{A}^{H_L}(\text{st})$ 
7:   if  $\text{Query} = (\text{Prove}, x, w)$  then
8:     if  $R(x, w) = 1$  then
9:        $\Pi \leftarrow \Pi_R^{\text{SLC}}.\text{SimProve}^{\text{Prog}_L}(z, x)$ 
10:      Append  $(x, \pi)$  to  $\text{pflist}$ 
11:      Set  $\text{Response} \leftarrow (x, \pi)$ 
12:     end if
13:   else if  $\text{Query} = (\text{Challenge}, x, \pi)$  then
14:     if  $\Pi_R^{\text{SLC}}.\text{Verify}^{\text{Prog}_L}(x, \pi) = 1$  and  $(x, \pi) \notin \text{pflist}$  then
15:        $w \leftarrow \Pi_R^{\text{SLC}}.\text{Extract}(x, \pi, \mathcal{Q}_{\mathcal{A}})$ 
16:       if  $R(x, w) = 0$  then
17:         return Fail
18:       end if
19:     end if
20:   end if
21: end while
22: return Success

```

Game 1: The non-interactive special simulation-soundness game.

B Glossary

Variable	Description
κ	Security parameter; length of the random oracle output.
T	The target to successfully solve a PoW.
m	The size of enclave public key that clients should encrypt to; $m = \Theta(\log^2 \kappa)$.
k	The common prefix parameter; $k = \Theta(\log^2 \kappa)$. When clients issue a transaction tx , they associate tx with the block on the tip of $\mathcal{C}_{\text{loc}}^{[k]}$.

R	The recency parameter of profile blocks.
K	The size of a window that is used to decide transaction timestamp, counted by the number of blocks.
h_r	Number of RO queries made by honest parties at round r .
t_r	Number of RO queries made by corrupted parties at round r .
δ	Advantage of honest parties ($t \leq (1 - \delta)h$).
f	Block generation rate; the probability at least one honest party succeeds in finding a PoW in a round.

Table 1: *Main parameters of $\Pi_{\text{Ledger}}^{\text{Fair}}$.*